
xLSTM Architecture’s Feasibility For Recommendations (Draft Version)

Vivekanand

Abstract

The primary idea of Recommender system is to present users with the most relevant items such as Music, Movies, Software, Services by learning large-scale interaction datasets. In this research paper, we will investigate the feasibility of extended Long Short-Term Memory (xLSTM) architecture for sequential recommendation tasks. We benchmark xLSTM against strong existing baselines, including Transformer-based models (such as BERT4Rec, SAS4Rec) and recurrent alternatives, on five public datasets: MovieLens-100K, MovieLens-1M, MovieLens-10M, MUSIK4all, and Amazon Software. We also evaluate and compare models using top-k ranking metrics (Recall@10, MRR@10, NDCG@10) with training time.

Our results show that, xLSTM attains competitive accuracy relative to Transformer baselines, particularly on medium and large-scale datasets, and while offering favorable memory scaling and stable training. On smaller datasets, performance gaps narrow or favor Transformers models, highlighting trade-offs between capacity and data sequence settings.

Overall, our results indicate that xLSTM is a strong sequence model at scale competitive with Transformer baselines on larger datasets like ML-1M/10M, memory-efficient and training-stable, yet less effective on the smallest dataset (ML-100K). Finally, these results and findings will point to a practical path for deploying xLSTM in modern large-scale recommender systems.

1 Motivation

The rapid growth of the user interaction data has created an unprecedented demand to create efficient recommender systems that must act faster, accurate and computationally efficient. Despite there are several advancements in the sequential recommendations research which are primarily based on existing architectures like LSTM and Transformer-based, but still the scalability, long-term dependency and computational problems remain the key challenges. This motivates to the exploration of the latest and enhanced architectures like xLSTM and to address the existing limitations and to check the feasibility of sequential recommendation research for the large scale interaction datasets.

The critical subdomain of recommenders is sequential recommendations, which models user-item interactions over the period of time, capturing temporal patterns and evolving user preferences. Recent advancements in this research area have leveraged deep learning architectures, including Recurrent Neural Networks (RNN) Elman [1990], Long Short Term Memory (LSTM) Hochreiter and Schmidhuber [1997], and other recent architectures like BERT4Rec Sun et al. [2019] and SASRec ?, which primarily adopt Transformer-based self-attention mechanisms for sequence modeling.

In this report, we will investigate xLSTM, primarily mLSTM Beck et al. [2024], an enhanced variant of LSTM that incorporates architectural improvements including bidirectional processing, attention integration, and refined gating mechanisms. We will also benchmark xLSTM against classical and state-of-the-art sequential recommenders using datasets like MovieLens and other temporal interaction datasets. Our final results shows that xLSTM demonstrates notable improvements due to

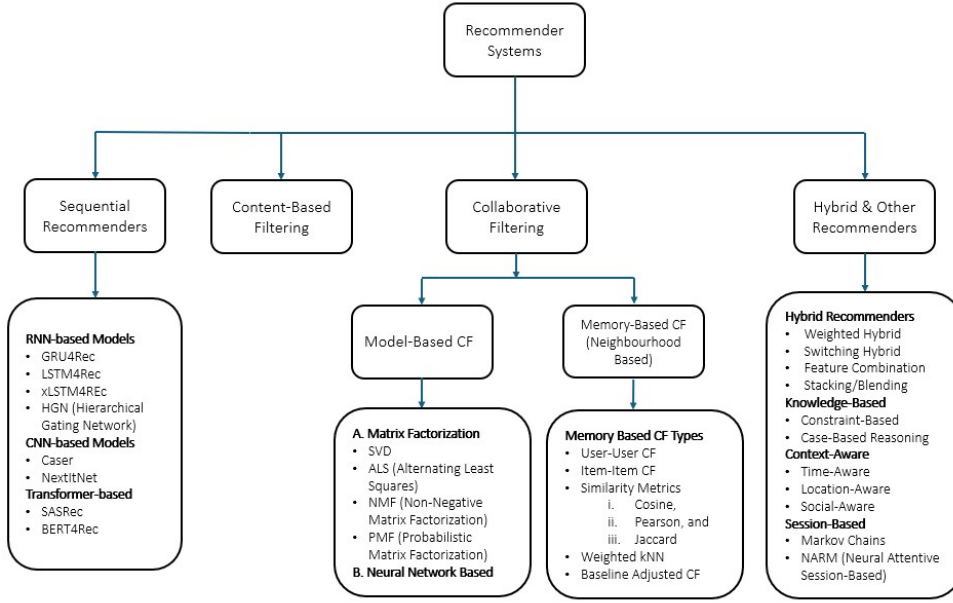


Figure 1: Recommenders Classifications

its hybrid design like mLSTM and sLSTM that blends recurrent memory and attention mechanisms, which we will leverage further.

1.1 Existing Architecture Overview

In earlier recommendation prediction tasks, Recurrent Neural Networks (RNN), Long Short Term Memory (LSTM) were commonly used to predict the end user preferences and provide us with personalized suggestions. Roy and Dutta [2022], Castells and Jannach [2023]. Classical methods such as collaborative filtering and content-based filtering will have also evolved to incorporate sequential modeling, capturing dynamic user behavior over the period of time Vats et al. [2024].

Among the deep learning approaches, Long Short-Term Memory (LSTM) network remains a widely used architecture for modeling sequential interactions due to its ability that it can preserve long-term dependencies and manage temporal dynamics effectively utilizing its gating mechanisms.

Despite its success in implementation, LSTM also suffered from several well-known limitations, such as: (1) its inability to revise storage decisions once made - which means once the LSTM decides whether to keep or forget some information, it cannot go back and change that decision later (2) the need to compress all contextual information into scalar cell states - which means that all the memory must be compressed into single vector, a fixed size memory, and (3) limited parallelizability due to its recurrent memory mixing Beck et al. [2024] - it can process one step at a time and hard to perform computations in parallel. These limitations have led to development of enhanced variants, such as xLSTM, which will introduces architectural refinements like attention mechanisms, bidirectional memory, and improved gating functions to better handle large scale sequential recommendation tasks.

1.2 General Classification of Recommender Systems

According to the RecBole framework RecBole [2020], recommender systems can be broadly classified into four major categories based on the nature of data used and task formulation:

1. **General Recommendation (GR):** These models rely solely on user-item interaction data, typically implicit feedback. Implicit feedback will contains the datasets which indirectly indicate user preferences such as clicks, Add-to-cart events, purchase, Time spent, frequency etc. They are evaluated through top- N recommendation tasks and primarily align with collaborative filtering approaches.

2. **Content-Aware Recommendation:** These models incorporate additional side information such as user or item features. They are often applied in click-through rate (CTR) prediction tasks, using explicit feedback and binary classification evaluation. Its feature based, often go beyond raw user-item interactions about users, items or context.
3. **Sequential Recommendation (SR):** SR focuses on next-item prediction by modeling the temporal ordering of user interactions. These models utilize sequential data to capture evolving user preferences. Session-based recommendations are generally included in this category and this is our primary research focus.
4. **Knowledge-Based Recommendation:** These methods leverage external knowledge graphs to enhance general or sequential recommendations by providing semantic or structural context beyond user-item interactions.

Broadly, recommendation systems can also be categorized into two paradigms: Collaborative Filtering, which models similarities between users or items based on interaction histories, and Content-Based Filtering, which relies on item attributes or metadata to make predictions Vats et al. [2024], Castells and Jannach [2023]

1.3 Problem Statement

Despite the advancements brought by deep learning in improving recommendation accuracy, several critical challenges persist in modeling user-item sequences at scale. These challenges include the cold-start problem, data sparsity, and the need for robust long-term dependency tracking across user interactions Noorian et al. [2024], Roy and Dutta [2022].

Recurrent architectures such as LSTM, though effective in capturing temporal dependencies, suffer from limited scalability and are inherently sequential, making them less efficient for large-scale recommendation environments Yang and Esquivel [2024], Ahmadian Yazdi et al. [2024]. In contrast, Transformer-based models like BERT4Rec and SASRec offer greater parallelism and improved performance through self-attention mechanisms but often come with high computational costs and memory requirements Sun et al. [2019], ?.

This work investigates whether the recently proposed xLSTM architecture Beck et al. [2024], which integrates bidirectional memory, attention mechanisms, and enhanced gating structures, can serve as a middle ground—achieving scalability, computational efficiency, and high accuracy in sequential recommendation tasks.

1.4 Research Questions and Contributions

This report aims to answer the following four key research questions:

- **RQ1:** How does xLSTM’s performance scale with dataset size compared to established architectures like BERT4Rec and Sas4Rec ?
- **RQ2:** How do sequence length and embedding size influence model performance across different item-popularity levels, and do larger sequences or embeddings improve a model’s ability to make accurate long-tail (less popular) recommendations?
- **RQ3:** What trade-offs exist between recommendation accuracy and computational cost as sequence length and model complexity increase?
- **RQ4:** Embedding Saturation and Utilization: How different model architectures make effective use of their embedding representations, and does model embedding dimensionality lead to better spatial distribution of embeddings, representation diversity, or predictive performance?

The primary objective is to answer these four RQ’s and evaluate the effectiveness of the xLSTM model across multiple datasets and benchmark it against state-of-the-art baselines using established ranking metrics.

The goal of this work is to perform next-item prediction by modeling sequential user–item interactions within a personalized recommendation framework. We also make following key contributions to the field of sequential recommender systems:

Implementation and Evaluation of xLSTM for sequential recommendation: We adapt the xLSTM model for next-item recommendation and implement it using user–item interaction data. We describe the main architectural choices needed to apply xLSTM in practical recommender system settings Beck et al. [2024]. We also present a novel adaptation, and configuring the latest available xLSTM model, detailing the architectural choices needed for practical deployment on user–item interaction data.

Comprehensive Model Benchmarking: We evaluate xLSTM across five public datasets and compare it with leading sequential models, including SAS4Rec and BERT4Rec Sun et al. [2019]. The comparisons use standardized top-k ranking metrics and controlled experimental settings, addressing performance scaling (RQ1) and sequence-length effects (RQ2).

Analysis of computational efficiency and scaling behavior: We examine model stability, training time, inference characteristics, and resource consumption to understand accuracy–efficiency trade-offs (RQ3), particularly under long sequences and large item movie predictions.

Embedding utilization and representation analysis: We conduct an in-depth study of embedding saturation, dimensionality behavior, and latent-space organization across models, offering new insights into how different architectures use embedding capacity (RQ4). Also, discuss interaction sparsity, long-range dependency modeling, and popularity imbalance.

2 Existing Model Architectures For Sequential Recommenders

2.1 Singular Value Decomposition (SVD)

SVD (matrix factorization) compresses the user–item matrix into small “latent factors.” Users and items get vectors; recommendations come from matching these vectors. It scales well and works great for steady preferences, but it ignores sequence dynamics and has trouble with cold-start users/items.

SVD breaks the large user–item matrix $\mathbf{R} \in \mathbb{R}^{m \times n}$ into three smaller pieces and keeps only the most informative parts:

$$\mathbf{R} \approx \mathbf{U}_k \Sigma_k \mathbf{V}_k^\top,$$

so each user and each item get a k -dimensional vector that captures their main tastes and properties Sarwar et al. [2000]. We usually work with matrix-factorization form, scoring user u and item i by

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i,$$

and learn $\mathbf{p}_u, \mathbf{q}_i$ by minimizing a regularized squared loss

$$\min_{\{\mathbf{p}, \mathbf{q}, b\}} \sum_{(u,i) \in \mathcal{D}} (r_{ui} - \mu - b_u - b_i - \mathbf{p}_u^\top \mathbf{q}_i)^2 + \lambda \left(\sum_u \|\mathbf{p}_u\|^2 + \sum_i \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2 \right),$$

which gives robust, low-rank representations and strong top- k recommendations. With implicit logs (clicks, plays) we weight observations via confidences c_{ui} instead of using star ratings directly, which makes the model usable at web scale Hu et al. [2008]. On its own, SVD is *static*: it explains long-term preference but ignores the order of events. To make it sequence-aware, one simple step is to let parameters drift over time (timeSVD++), e.g., time-dependent biases $b_u(t), b_i(t)$ and decays for older interactions, so recent behavior counts more Koren [2009a]. Another route is to factorize *transitions*: model the probability of the next item from the last one and the user. FPMC combines MF with a first-order Markov chain and scores a candidate by user taste *and* compatibility with the last item, yielding strong next-item predictions Rendle et al. [2010]. In practice we also train on sliding windows or apply exponential decay before factorization to emphasize recency. Because it is fast, stable, and easy to tune, SVD serves as a solid baseline for sequential tasks, a candidate generator for heavier models, and a source of clean embeddings to initialize LSTMs/Transformers.

2.2 Recurrent Neural Networks (RNN)

Recurrent Neural Networks read a user’s history in order (one step after another) to predict the next item. They capture short-term trends (what the user is into right now), but training can be unstable and slow on long sequences.

Recurrent Neural Networks (RNN). A (vanilla) RNN updates its hidden state by combining the current input with the previous state, then predicts the next item; it is simple and fast but struggles with long-range dependencies due to vanishing gradients Elman [1990], Rumelhart et al. [1986], Bengio et al. [1994], Pascanu et al. [2013].

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b_h),$$

$$s_{t+1} = E^\top h_t,$$

$$\hat{y}_{t+1} = \text{softmax}(s_{t+1}),$$

$$L = - \sum_t \log \hat{y}_t[y_t] \quad (\text{cross-entropy over the true next item } y_t),$$

Where x_t is the current input (item embedding), h_t the hidden state, $\phi(\cdot)$ a pointwise nonlinearity (e.g., tanh or ReLU), E the shared item-embedding/output matrix, and W_x, W_h, b_h learned parameters. Training uses backpropagation through time (BPTT) Rumelhart et al. [1986].

2.3 Long Short Term Memory (LSTM)

LSTM Hochreiter and Schmidhuber [1997] enhanced sequential learning further by remembering what standard RNNs forgot, making it a cornerstone for early sequence-aware recommenders in the previous section. LSTM serves as the architectural predecessor to xLSTM, directly informing RQ1 (scaling) and RQ2 (sequence length). LSTM is an advanced RNN variant with better gates controlling information flow to capture long-range dependencies and mitigate vanishing gradient issues Hochreiter and Schmidhuber [1997] (previously discussed limitations with RNN). Here, LSTM improves on RNNs with gates that help the model to remember important past interactions for longer. It's a strong sequential baseline Hochreiter and Schmidhuber [1997] model used widely before the Transformers, offering better long-range memory.

However, it can be heavier to train and may still lag behind attention models on very long histories. LSTMs extend RNNs with a gated memory cell that preserves information over long spans, mitigating vanishing gradients and enabling next-item predictors to remember earlier parts of a user's sequence. Hochreiter and Schmidhuber [1997]

LSTM has three gates in its design: 1. Forget gate which decides what information to discard from the cell state, 2. Input gate which determines what new information to add to the cell state and Output Gates which controls what information is output from the cell state as the hidden state Hochreiter and Schmidhuber [1997]. Sigmoid activation in the gates (forget, input, output) to control the flow between 0 and 1, and tanh activation for updating the cell state and output.

The range of all three gates is between 0 and 1, due to sigmoid activation function, which outputs values in the range (0,1). The values determine how much information passes through each gate, where 0 means "block all information," and 1 means "allow all information" Hochreiter and Schmidhuber [1997].

Element-wise multiplication (also called the Hadamard product) in its gating mechanisms. This means that each element of the gate vector (values between 0 and 1 from the sigmoid activation) multiplies the corresponding element of another vector (such as the previous cell state or the candidate cell update). this operation allows gates to selectively scale each component of the cell state or hidden state, controlling how much information to keep or discard for each dimension independently Hochreiter and Schmidhuber [1997].

LSTMs add a gated memory cell to overcome vanishing gradients and keep long-range context in user sequences Hochreiter and Schmidhuber [1997], Gers et al. [2000].

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i),$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f),$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o),$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c).$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

$$h_t = o_t \odot \tanh(c_t).$$

$$\begin{aligned}s_{t+1} &= E^\top h_t, \\ \hat{y}_{t+1} &= \text{softmax}(s_{t+1}).\end{aligned}$$

The weight matrices $W_i, W_f, W_o, W_c \in \mathbb{R}^{d \times d_x}$ correspond to the input-to-hidden connections between the current input vector $x_t \in \mathbb{R}^{d_x}$ and the input gate, forget gate, output gate, and candidate cell state, respectively. The matrices $U_i, U_f, U_o, U_c \in \mathbb{R}^{d \times d}$ denote the recurrent (hidden-to-hidden) connections between the previous hidden state $h_{t-1} \in \mathbb{R}^d$ and the respective gates and cell input. The bias terms $b_i, b_f, b_o, b_c \in \mathbb{R}^d$ are learned vectors. The functions $\sigma(\cdot)$ and $\tanh(\cdot)$ denote the logistic sigmoid and hyperbolic tangent activation functions applied element-wise, where the sigmoid gates control information flow and tanh squashes values to a bounded range. The cell state $c_t \in \mathbb{R}^d$ is a vector that acts as the network’s memory, enabling preservation of long-term information, while the hidden state $h_t \in \mathbb{R}^d$ is a vector representing the exposed output used for prediction. Through gated additive updates, LSTM mitigates the vanishing gradient problem of vanilla RNNs and allows more stable learning over long interaction sequences. $E \in \mathbb{R}^{N \times d}$ denotes the shared item embedding (output) matrix, where each row corresponds to an item representation used to score all candidate items for next-item prediction. $y_t \in \{1, \dots, N\}$ is the ground-truth next item index (scalar) and $\odot$ denotes element-wise multiplication. The output head compares h_t with E to produce next-item probabilities for ranking Hochreiter and Schmidhuber [1997].

Although LSTM does not impose a specific loss function, we can employ a softmax cross-entropy objective appropriate for next-item prediction, which is consistent with prior sequential recommendation work. For next-item recommendation, we can use a softmax cross-entropy objective that encourages the model to assign high probability to the true next item in the sequence while suppressing all other candidates Hochreiter and Schmidhuber [1997].

$$\begin{aligned}s_{t+1} &= E^\top h_t, \\ p(y_{t+1} | h_t) &= \text{softmax}(s_{t+1}), \\ \mathcal{L} &= - \sum_{t=1}^T \log p(y_{t+1}^* | h_t),\end{aligned}$$

Where $h_t \in \mathbb{R}^d$ is the sequence representation at time t (vector), $E \in \mathbb{R}^{N \times d}$ is the item embedding/output matrix (matrix), $s_{t+1} \in \mathbb{R}^N$ is the unnormalized score vector for all items (vector), $p(y_{t+1} | h_t) \in \mathbb{R}^N$ is the predicted probability distribution (vector), $y_{t+1}^* \in \{1, \dots, N\}$ is the ground-truth next item index (scalar), T is the sequence length, and $\mathcal{L}$ is the training loss (scalar).

For a sequential datasets like MovieLens, with an interaction sequence $\langle \text{ItemID}_1, \text{ItemID}_2, \dots, \text{ItemID}_t \rangle$, the model processes the embeddings of the observed items to produce a sequence representation h_t . Using this representation, it computes a score vector s_{t+1} for all items in the catalog. The softmax function converts these scores into a probability distribution Hochreiter and Schmidhuber [1997] over item IDs, and the training loss penalizes the model if the true next movie ItemID_{t+1} is assigned low probability. By minimizing the loss $\mathcal{L}$ across all users and time steps, the model learns to rank the correct next item higher than other candidates.

LSTM’s memory bottlenecks Hochreiter and Schmidhuber [1997] are closely tied to RQ1 (scaling) and RQ2 (sequence length effects). The main limitations of LSTMs in sequential recommendations are: difficulty capturing very long-term dependencies accurately, high computational cost with a slower training on large sequential datasets, and challenges in the interpretability and hyperparameter sensitivity, which we will address in the upcoming methods.

2.4 Self-Attentive Sequential Recommendation (SAS4Rec)

SAS4Rec Kang and McAuley [2018] originally uses the self-attention mechanisms to effectively capture long-term user patterns or context in sequential recommendation tasks, which is important for evaluating competitiveness in RQ1 and RQ3. Unlike the traditional Markov Chains and RNN-based approaches, SAS4Rec adaptively focuses on the most relevant items in a user’s history, allowing it to balance long-range dependency modeling with computational efficiency. The architecture employs Transformer-style self-attention blocks, positional embeddings to preserve the order of items, and incorporates residual connections and layer normalization to stabilize training. SAS4Rec outperforms state-of-the-art sequential models on both sparse and dense datasets, demonstrating the better accuracy and being significantly faster due to its parallelizable design similar to BERT4Rec (which is discussed in the next section). The model dynamically adjusts its attention span depending on dataset density,

focusing narrowly on recent actions in sparse data and broadly on several past actions in denser data Kang and McAuley [2018]. The visualizations of attention weights provides insights into how the model captures meaningful sequential patterns that drive recommendations. This approach marks a significant advancement in efficient, interpretable, and accurate sequential recommendation systems.

SAS4Rec models user sequences using masked self-attention. Given the sequence embedding matrix $E_b \in \mathbb{R}^{L \times d}$, the self-attention operation is defined as $\text{SA}(E_b) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}} + \mathbf{M}\right) V$,
 $Q = E_b W^Q$, $K = E_b W^K$, $V = E_b W^V$,

Where $\mathbf{M}$ is a causal mask that prevents attending to future positions, $E_b \in \mathbb{R}^{L \times d}$ denotes the sequence embedding matrix for a user, with sequence length L and embedding dimension d . The matrices $W^Q, W^K, W^V \in \mathbb{R}^{d \times d}$ are learned projection matrices that map input embeddings to the query, key, and value spaces, respectively. The resulting matrices $Q, K, V \in \mathbb{R}^{L \times d}$ represent the query, key, and value vectors for all positions in the sequence. The term $QK^\top \in \mathbb{R}^{L \times L}$ computes pairwise similarity scores between all item positions, which are scaled by $\sqrt{d}$ for numerical stability. The matrix $\mathbf{M} \in \mathbb{R}^{L \times L}$ is a causal (lower-triangular) mask that assigns $-\infty$ or a very large negative number to future positions, ensuring that each item attends only to past interactions. The $\text{softmax}(\cdot)$ function is applied row-wise to produce attention weights, and the final output $\text{SA}(E_b) \in \mathbb{R}^{L \times d}$ is a matrix of context-aware item representations used for next-item prediction Kang and McAuley [2018].

The relevance score between item i and the hidden representation at time t is computed as $r_{i,t} = \mathbf{h}_t^{(b)} \mathbf{M}_i^\top$, where $\mathbf{h}_t^{(b)}$ is the output of the final self-attention block and $\mathbf{M}_i$ denotes the item embedding.

SAS4Rec is trained using a binary cross-entropy loss with negative sampling: $\mathcal{L} = - \sum_{t=1}^n \left[\log \sigma(r_{o_t,t}) + \sum_{j \in \mathcal{N}_t} \log \sigma(-r_{j,t}) \right]$,

Where t indexes the position in the user interaction sequence, $o_t \in \{1, \dots, N\}$ is the ground-truth next item at position t (scalar item ID), $\mathcal{N}_t$ is a set of sampled negative item indices (scalars), and $\sigma(\cdot)$ denotes the logistic sigmoid function. The score $r_{i,t} \in \mathbb{R}$ is a scalar relevance score for item i at time t , computed as $r_{i,t} = \mathbf{h}_t^\top \mathbf{e}_i$, where $\mathbf{h}_t \in \mathbb{R}^d$ is the context-aware hidden representation produced by the masked self-attention layer, and $\mathbf{e}_i \in \mathbb{R}^d$ is the embedding vector of item i . This objective encourages high scores for the true next item while suppressing scores for sampled negative items Kang and McAuley [2018].

Transformer-style self-attention to the recommendation domain, which models user-item sequences without recurrence, allowing for full parallelism and effective long-range dependency capture Kang and McAuley [2018]. The model uses positional encodings and attention weights to identify the most relevant past items when predicting the next interaction, offering high accuracy and scalability, analyzed in RQ1 and RQ3.

2.5 Bidirectional Encoder Representations for Recommendation (BERT4Rec)

BERT4Rec adopts the BERT (Bidirectional Encoder Representations from Transformers) Sun et al. [2019] architecture (figure 2.1) for the sequential recommendation. This acts as the strongest Transformer baseline in this report, critical for benchmarking xLSTM (RQ1, RQ2) and comparing embedding behavior (RQ3, RQ4). It treats user interaction data as a sequence and uses a masked item prediction task to learn the bidirectional dependencies. Unlike SAS4Rec, which processes in a left-to-right fashion, here BERT4Rec learns from both past and future items, improving its ability to model complex sequence semantics.

RNN and SAS4Rec which encode users/items historical interactions from left to right into hidden representations for the predictions Elman [1990] Kang and McAuley [2018]. Such unidirectional model architectures restrict the power of historical sequence representations and in order to address this challenge which BERT4Rec will address this Sun et al. [2019].

Conditioning both left and right contexts in a deep bidirectional model can trivialize training because each item may indirectly access the target item in the sequence. To address this, a bidirectional

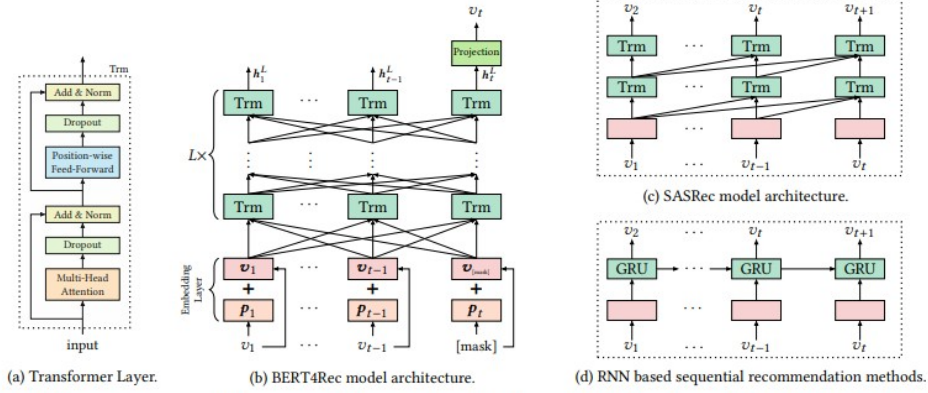


Figure 1: Differences in sequential recommendation model architectures. BERT4Rec learns a bidirectional model via Cloze task, while SASRec and RNN based methods are all left-to-right unidirectional model which predict next item sequentially.

Figure 2: Bert4Rec Model Architecture

model is trained using the Cloze task Sun et al. [2019], which predicts randomly masked items in the sequence based on both their left and right contexts. Unlike predicting the next item sequentially, the Cloze task generates more training samples, enabling the model to learn a stronger representation and achieve consistently better performance on benchmark datasets. Sun et al. [2019]

BERT4Rec models user behavior using bidirectional self-attention and a masked-item prediction objective. Its core components are defined as follows:

1. **Scaled dot-product self-attention (context mixing across the sequence).** Attention(Q,K,V) = $\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$,

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V,$$

Where $H \in \mathbb{R}^{L \times d}$ is the matrix of hidden representations for a length- L interaction sequence, with each row $\mathbf{h}_t \in \mathbb{R}^d$ corresponding to the contextual embedding of the item at position t . The matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ are learned projection matrices that map hidden states to query, key, and value spaces, respectively. The resulting matrices $Q, K, V \in \mathbb{R}^{L \times d_k}$ contain query, key, and value vectors for all positions in the sequence. The term $QK^\top \in \mathbb{R}^{L \times L}$ computes pairwise similarity scores between all items in the sequence, scaled by $\sqrt{d_k}$ for numerical stability. The softmax function is applied row-wise to produce attention weights that sum to one, enabling each position to attend to all other positions in the sequence. The output of the attention layer is a matrix in $\mathbb{R}^{L \times d_k}$ representing context-aware item embeddings used in subsequent Transformer layers Sun et al. [2019].

2. **Masked-position prediction with tied item embeddings.** $P(v \mid S', t) = \text{softmax}(\mathbf{h}_t^{(L)} E^\top + \mathbf{b})$,

Where $\mathbf{h}_t^{(L)} \in \mathbb{R}^d$ is the hidden representation produced by the final Transformer layer at the masked position t , encoding contextual information from the entire sequence S' . The matrix $E \in \mathbb{R}^{N \times d}$ denotes the shared item embedding matrix, where each row corresponds to the embedding of one item in the catalog of size N ; this matrix is tied with the input embedding layer. The bias vector $\mathbf{b} \in \mathbb{R}^N$ is a learned parameter that accounts for item-specific popularity effects. The product $\mathbf{h}_t^{(L)} E^\top \in \mathbb{R}^N$ produces a score for every item, and the softmax function converts these scores into a probability distribution $P(v \mid S', t)$ over all items, representing the likelihood that item v is the correct masked item at position t Sun et al. [2019].

3. **Cloze (masked-item) training objective.** $\mathcal{L} = -\frac{1}{|M| \sum_{t \in M} \log P(v_t^* \mid S', t)}$,

Where $M \subset \{1, \dots, T\}$ denotes the set of masked positions in the input sequence S' , and $|M|$ is the number of masked items. The symbol $v_t^* \in \{1, \dots, N\}$ represents the

ground-truth item index (scalar) at masked position t , drawn from a catalog of N items. The conditional probability $P(v_t^* | S', t)$ is obtained from the softmax output of the Transformer encoder at position t . This objective maximizes the likelihood of recovering the true masked items given their surrounding context, enabling bidirectional sequence modeling without relying on left-to-right recurrence Sun et al. [2019].

We will use Bert4Rec as a strong baseline model to compare its performance Sun et al. [2019] against SAS4Rec and xLSTM in upcoming sections. BERT4Rec is the strongest Transformer baseline in our study, critical for evaluating xLSTM’s competitiveness (RQ1). Its embedding behavior provides a meaningful comparison point for RQ4, especially regarding saturation and representational geometry.

3 Existing Literature Review and Architectures

To understand where xLSTM fits within the broader evolution of sequential recommenders, we review prior work that blends sequence modeling, attention, LSTMs, and Transformers. Each of these approaches exposes limitations or open questions that directly motivate the research direction of this report. This Existing literature review is structured around the four research questions (discussed in section 1.4) thereby motivating the design choices and evaluation focus of our proposed xLSTM-based approach. Each paragraph synthesizes prior work relevant to one research question and highlights the remaining gaps.

Performance scaling across dataset sizes (RQ1), early sequential recommender systems predominantly relied on recurrent neural architectures such as LSTM to model temporal dependencies in user–item interactions. Works such as noorian2024bertlstm and yang2024businesslstm DynaPR framework demonstrated that LSTM-based models can effectively capture evolving preferences on moderate-sized datasets, but their performance degrades as interaction volume grows due to limited parallelism and sequential computation. Educational and domain-specific systems based on BiLSTM further highlight these scalability constraints when applied to large or heterogeneous datasets Ahmadian Yazdi et al. [2024]. These findings motivate RQ1, which investigates whether xLSTM can better maintain performance as dataset size increases compared to classical LSTM and Transformer-based baselines.

Sequence-length sensitivity and long-tail behavior (RQ2), Several studies emphasize modeling recent interactions using attention-enhanced recurrent architectures, particularly in session-based recommendation. kumar2024sessionattention show that attention-driven LSTMs improve short-term intent modeling but struggle with long user histories and sparse item interactions. Transformer-based models such as SAS4Rec and BERT4Rec address this by modeling the global sequence context through self-attention Kang and McAuley [2018], Sun et al. [2019]. However, their effectiveness varies significantly with sequence length and data density, often favoring users with long, dense histories and popular items. These observations directly inform us the focus for RQ2, which examines how sequence length and embedding capacity influence performance across different item-popularity regimes, particularly for long-tail recommendations.

Accuracy–efficiency trade-offs perspective (RQ3), While Transformer-based architectures deliver strong accuracy, their quadratic computational complexity (it means how the time and memory cost grow with sequence length, for a sequence of length L and embedding dimension d , self-attention in Transformer-based models has time complexity $\mathcal{O}(L^2d)$ and memory complexity $\mathcal{O}(L^2)$ due to the $L \times L$ attention matrix, whereas recurrent models scale linearly as $\mathcal{O}(Ld)$.) with respect to sequence length imposes significant memory and runtime costs. In other words, Transformer has to compare every item in a user’s history with every other item at each step. As the sequence gets longer, the amount of computation and memory grows very fast—if the history length doubles, the cost becomes about four times larger in general. Generative retrieval models such as TIGER Rajput et al. [2024] further increase computational overhead by combining Transformer encoders with large output spaces, limiting their practicality in large-scale or low-latency environments. Conversely, recurrent models offer lower per-step complexity but suffer from slower training and limited scalability. Surveys of large language model–based recommenders also highlight the inefficiency of applying general-purpose attention-heavy models to sequential recommendation tasks Vats et al. [2024]. These contrasting properties motivate RQ3, which analyzes the trade-offs

between recommendation accuracy and computational cost as sequence length and model complexity increase.

Beyond accuracy, representation quality and embedding utilization (RQ4), beyond predictive performance, recent literature highlights structural issues in learned embedding spaces, including anisotropy, popularity-driven compression, and limited effective dimensionality. Transformer-based recommenders have been shown to concentrate variance into a small number of dominant directions, leading to hubness and reduced long-tail diversity Gheewala et al. [2023]. Although recurrent architectures distribute information more gradually, their limited expressiveness restricts representation diversity in complex sequences. These findings motivate RQ4, which investigates how different architectures utilize embedding capacity, whether higher-dimensional embeddings translate into latent spaces Beck et al. [2024], and how xLSTM compares to Transformer baselines in terms of representation diversity and stability.

Taken together, prior work reveals persistent limitations across scalability (RQ1), long-sequence robustness (RQ2), efficiency–accuracy trade-offs (RQ3), and embedding utilization (RQ4). These gaps suggest the need for an architecture that combines recurrent memory with attention mechanisms while remaining computationally efficient and representationally stable. The xLSTM architecture is designed to address these challenges Beck et al. [2024] and is therefore evaluated in this report with respect to RQ1–RQ4.

4 Methodology

This section outlines the experimental design, data preparation procedures, model configurations, and evaluation protocols used to investigate the performance of the xLSTM model in sequential recommendation tasks.

4.1 Datasets

We conducted experiments on four benchmark datasets widely used in the recommender systems community:

- **MovieLens-100K, 1M, and 10M:** Publicly available under the GroupLens license Harper and Konstan [2015], enabling benchmarking under both sparse and dense user–item conditions.
- **MUSIK4all:** Large-scale music interaction dataset (228M events, 119k users) with play counts and timestamps. Released by Johannes Kepler University Linz for music recommendation research Schedl et al. [2022].
- **Amazon Product Data (Software subset):** Publicly available review and rating data capturing millions of user–item interactions. Provides timestamps and metadata for collaborative and content-based recommendation benchmarking. ?Ni et al. [2019].

Table 1: Experimental datasets.

Dataset	Users	Items	Interactions	Avg. len.	Sparsity
ML-100K	943	1,682	100,000	106.05	0.9369
ML-1M	6,040	3,416	999,611	165.49	0.9515
ML-10M	71,567	10,681	10,000,054	139.74	0.9869
Music For All	4584	51,291	2,533,266	552.6	0.9991
Amazon SW	3,75,147	21,663	4,59,436	-	0.9892

All datasets underwent preprocessing to ensure consistent format and indexing across experiments.

4.2 Model Pipeline

4.3 Model Selection

To further evaluate our research questions RQ1–RQ4, we need strong baseline models to compare against xLSTM. This subsection explains why we selected BERT4RecSun et al. [2019],

Recommender System Pipeline

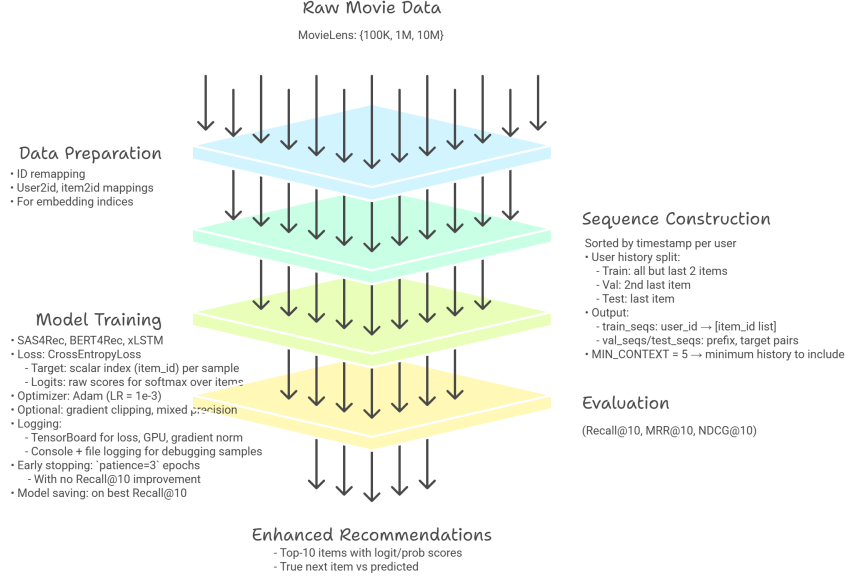


Figure 3: Model Pipeline

SAS4RecKang and McAuley [2018], and xLSTM Beck et al. [2024], as each represents a different approach to sequential recommendation. This selection enables a controlled comparison of scalability, sequence modeling capacity, embedding utilization, and computational efficiency against the proposed xLSTM architecture.

xLSTM is our proposed model architectureBeck et al. [2024], as it extends classical LSTM with enhanced gating, chunkwise memory updates, and matrix-based state representationsBeck et al. [2024]. These mechanisms are explicitly designed to address long-range dependency tracking and memory saturation while maintaining linear complexity in sequence length. Evaluating xLSTM directly tests whether recurrent architectures can close the performance gap with attention-based models on large sequential datasets (RQ1, RQ2) while offering improved efficiency and stability (RQ3).

SAS4Rec is selected as a strong unidirectional Transformer baselineKang and McAuley [2018]. By relying on causal self-attention, SAS4Rec models long-range dependencies without recurrence and represents the standard approach for next-item prediction under temporal constraintsKang and McAuley [2018]. Its quadratic attention cost makes it particularly relevant for analyzing accuracy–efficiency trade-offs as sequence length increases (RQ3), as well as embedding utilization under popularity-biased data (RQ4).

BERT4Rec as a bidirectional Transformer baseline Sun et al. [2019] that studies causality during training through masked-item prediction. This allows the model to learn the item–item relationships and serves as an upper-bound reference for representation quality and ranking accuracy Sun et al. [2019]. Comparing xLSTM against BERT4Rec highlights the impact of bidirectional context modeling on embedding geometry and long-tail performance (RQ2, RQ4), as well as the computational cost of full self-attention (RQ3).

Together, these models provide complementary baselines that isolate the effects of recurrence versus attention, unidirectional versus bidirectional encoding, and linear versus quadratic computational

scaling Beck et al. [2024]. This diversity of model selection is essential for a principled evaluation of xLSTM’s feasibility as a scalable and robust sequential recommender.

4.4 Data Preparation and Splitting Protocol

To preserve chronological integrity and ensure fair evaluation, the data was split at the user level using the following procedure:

- **Training Set:** All user interactions from timestep $t = 1$ to $t = N - 2$, excluding the last two items. For each user, the training set comprises all interactions from timestep $t = 1$ to $t = N - 2$, where N is the total number of interactions for that user. This configuration ensures the model learns from a substantial portion of the user’s historical behavior, allowing it to capture long-term preferences and sequence patterns while preventing exposure to future events (i.e., data leakage).
- **Validation Set:** The second-to-last item (i.e., at $t = N - 1$) used to tune model parameters and trigger early stopping. The interaction at timestep $t = N - 1$ is reserved for validation. This item is not seen during training and is used to monitor the model’s ability to generalize to unseen future behavior. Performance on the validation item drives hyperparameter tuning and early stopping criteria. By placing the validation set immediately after the training window, it closely mimics real-world next-item prediction scenarios.
- **Test Set:** The final item (i.e., at $t = N$) used for final evaluation of recommendation accuracy. The final interaction at timestep $t = N$ is used as the held-out test point for evaluating model performance. This setup reflects a realistic deployment environment where the goal is to predict the next most relevant item given all historical interactions up to the test moment. The use of a single-item prediction aligns with the sequential recommendation paradigm, ensuring a fair and consistent benchmark across users and models.
- **Sequence Lengths:** The models were trained and evaluated using varying sequence lengths of 32, 64, and 128. These values represent the maximum number of past user interactions considered when predicting the next item. By experimenting with multiple sequence lengths, we assess the model’s robustness and adaptability to different user history depths. Short sequences (e.g., 32) simulate scenarios with limited historical data, such as cold-start or session-based use cases. Medium and long sequences (e.g., 64 and 128) represent users with rich interaction histories, where capturing long-term dependencies becomes critical. This variation allows for analyzing how different architectures scale with memory depth and whether performance degrades or improves with longer input contexts.

4.5 Training Configuration

- **ID Remapping:** All user and item identifiers were remapped to continuous integer indices starting from zero. This remapping step ensures a compact and contiguous representation of categorical variables, which is essential for optimizing memory usage and improving the efficiency of embedding lookups during training. In large-scale datasets where raw IDs may be sparse or non-sequential (e.g., hashed strings or UUIDs), direct indexing can lead to large, sparse embedding tables with high memory overhead. Integer remapping also enables tight integration with matrix operations and GPU acceleration, which expect dense index ranges. This preprocessing step is standard in scalable recommender system pipelines and directly influences training stability and computational throughput [Rendle et al., 2020, Zhao et al., 2020].
- **Temporal Ordering:** All user interaction sequences were chronologically ordered to ensure that the model learns realistic user behavior patterns over time. This temporal consistency is crucial for sequential recommenders, which rely on the assumption that user preferences evolve as a function of prior interactions. By preserving the true order of events, the model is better equipped to capture short-term trends, session-based preferences, and long-range temporal dependencies. Chronological ordering also prevents data leakage during training and evaluation, which would otherwise inflate performance metrics and lead to unrealistic generalization [Hidasi et al., 2016, Quadrana et al., 2018].
- **Early Stopping:** To prevent overfitting and reduce unnecessary computation, training was terminated if Recall@10 on the validation set did not improve over three consecutive

evaluation checkpoints (patience = 3). This strategy ensures that the model does not over-optimize on the training set, particularly important in sequential recommendation tasks where validation performance is a more reliable indicator of generalization. Early stopping also helps in controlling training time and model complexity, as suggested in empirical best practices for deep learning in recommender systems [Srivastava et al., 2014, Zhang et al., 2021].

- **Reproducibility:** Experiments were conducted using three different random seeds (42, 123, and 2023), which is essential for ensuring statistical robustness in sequential recommendation tasks. Single-run evaluations may suffer from high variance due to randomness in initialization, data ordering, or optimizer behavior. By averaging results across multiple seeds, we ensure that observed performance trends are stable, reliable, and not artifacts of random fluctuations. This practice strengthens the scientific reproducibility of our findings, as recommended in recent reproducibility studies in machine learning [Raff, 2021, Hutson, 2018].

5 Model Architectures Compared

This study evaluates and compares the following model architectures:

xLSTM: The proposed architecture that extends traditional LSTM with chunkwise attention, bidirectional memory routing, and GPU-optimized kernels for faster training and inference.

GRU4Rec: A gated recurrent unit (GRU) based model tailored for session-based recommendation tasks. It serves as a lightweight baseline.

SASRec: A Transformer-based model that utilizes self-attention mechanisms to model user sequences without recurrence. Particularly effective for capturing long-range dependencies.

BERT4Rec: A bidirectional Transformer model that uses masked language modeling for sequential recommendation, allowing the system to learn item dependencies in both forward and backward directions.

Table 2: Model architecture comparison across key dimensions.

Feature	LSTM	xLSTM	BERT4Rec	SASRec
Attention	No	Yes	Yes	Yes
Bi-directional	No	Yes	Yes	No
Training Stability	Medium	High	Medium	High
Inference Speed	Fast	Moderate	Slow	Moderate

Table 2 presents a comparative analysis of architectural and operational characteristics across LSTM, xLSTM, BERT4Rec, and SASRec. The key takeaways are:

- **Attention Mechanism:** xLSTM, BERT4Rec, and SASRec employ attention mechanisms to capture contextual relevance in sequences, unlike standard LSTM which lacks this capability Sun et al. [2019], ?, Beck et al. [2024].
- **Bidirectionality:** Both xLSTM and Transformer-based models (BERT4Rec and SASRec) support bidirectional processing, enhancing their ability to learn dependencies from both past and future contexts. In contrast, traditional LSTM is unidirectional Beck et al. [2024].
- **Training Stability:** xLSTM offers improved training stability due to its gated memory structure and regularization-friendly design. This leads to more consistent convergence relative to both LSTM and BERT4Rec Deng et al. [2024], Wu et al. [2024].
- **Inference Speed:** LSTM remains the fastest during inference due to its simplicity. xLSTM strikes a middle ground with moderate inference speed, while BERT4Rec is typically slower due to the computational overhead of deep Transformer layers Sun et al. [2019].
- **Overall Trade-off:** xLSTM provides a balance between modeling complexity, interpretability, and runtime performance, making it a compelling alternative for applications where both sequential fidelity and efficiency matter Beck et al. [2024].

5.0.1 xLSTM

Proposed xLSTM Based Approach Recommender in Details

Proposed xLSTM-Based Recommender Architecture To address the limitations of traditional LSTM architectures in sequential recommendation tasks, we adopt the recently proposed xLSTM framework Beck et al. [2024]. xLSTM introduces two principal variants: *sLSTM*, which employs scalar memory updates and memory mixing strategies, and *mLSTM*, which utilizes matrix memory representations with a covariance-based update rule, allowing for greater parallelization and expressiveness.

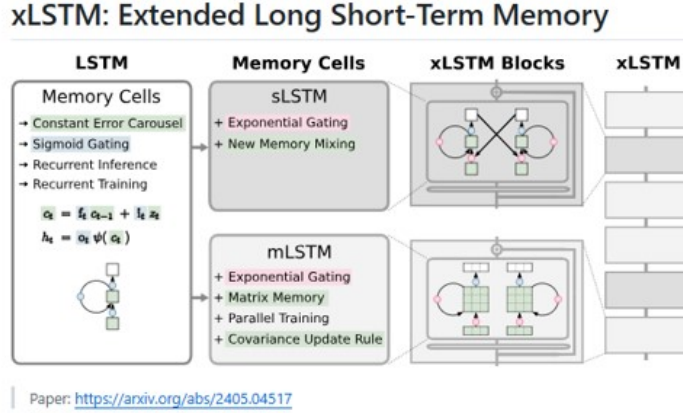


Figure 4: xLSTM Architecture

The architectural design of xLSTM enhances standard LSTM through several mechanisms that make it more suitable for modern recommender systems:

- **Extended Memory Capacity:** xLSTM integrates long-term memory structures that capture extended temporal dependencies, enabling it to retain and leverage long-range user behavior patterns more effectively than traditional RNNs.
- **Sparse Attention Mechanism:** Unlike fully-connected attention in Transformers, xLSTM employs a sparse attention scheme that dynamically focuses on the most relevant parts of the sequence, improving both interpretability and computational efficiency Wu et al. [2024].
- **Adaptive Gating Functions:** The model includes flexible gating mechanisms that adjust to input variability, allowing for more refined pattern extraction and robust sequence modeling across heterogeneous datasets Deng et al. [2024].
- **Bidirectional Processing and Parallelism:** xLSTM supports bidirectional sequence processing and is designed with parallelizable components, which facilitate scalable training and real-time inference—important features for production-grade recommender systems.

xLSTM is an extended version of the traditional LSTM (Long Short-Term Memory) network. It is designed to process sequential data more efficiently and accurately. Like standard LSTMs, it works with memory cells and gates to control the flow of information. However, xLSTM introduces new mechanisms to improve how information is stored and transferred. The key idea is the introduction of “cross-gates,” which connect hidden and memory states more dynamically and these cross-gates allow deeper interaction between different parts of the model. As a result, the network can better capture both short-term and long-term dependencies and this makes it especially effective for tasks involving long sequences, like sequential recommenders and time-series data. In normal LSTMs, gradients tend to vanish or explode over time, reducing performance. xLSTM reduces this issue by improving gradient flow across time steps. It also borrows ideas from Transformers, such as improved normalization and parameter efficiency. This gives xLSTM the ability to compete with large Transformer models while using fewer resources. It’s more memory-efficient and faster to train compared to attention-based networks. xLSTM also introduces better control over internal

information routing. This means it can learn more meaningful representations from complex data. The architecture is flexible, allowing it to adapt to various sequence learning problems. It can handle natural language, speech signals, and other temporal patterns effectively. The cross-gating mechanism acts like a bridge between past and present information. This helps the model maintain a context while predicting future outputs. In practice, xLSTM achieves higher accuracy on benchmark datasets compared to vanilla LSTMs. Its design allows deeper networks without losing stability during training. By combining simplicity with advanced gating, it achieves strong generalization performance. It avoids the heavy computational cost that comes with self-attention mechanisms. Thus, xLSTM provides a balanced trade-off between LSTM’s interpretability and Transformer’s power. It preserves the time-aware nature of RNNs while modernizing their inner workings. The result is a model that learns faster, remembers longer, and performs smarter. xLSTM is not just a minor upgrade — it’s a structural evolution of LSTM networks. In simple words, xLSTM blends the strengths of memory-based models and attention mechanisms. This makes it a robust, efficient, and future-ready architecture for deep learning on sequences.

In practice, xLSTM achieves higher accuracy on benchmark datasets compared to vanilla LSTMs Beck et al. [2024]. Its design allows deeper networks without losing stability during training. By combining simplicity with advanced gating, it achieves strong generalization performance. It avoids the heavy computational cost that comes with self-attention mechanisms. Thus, xLSTM provides a balanced trade-off between LSTM’s interpretability and Transformer’s. It preserves the time-aware nature of RNNs while modernizing their inner workings. The result is a model that learns faster, remembers longer, and performs smarter.

This makes it a robust, efficient, and future-ready architecture for deep learning on sequences.

[leftmargin=*]

1. **Exponential input gate (new gating):** grows multiplicatively to let the cell store strong events. $i_t = \exp(\tilde{i}_t)$, $\tilde{i}_t = w_i^\top x_t + r_i h_{t-1} + b_i$
2. **Forget/output gates (optionally exponential):** control decay and exposure of memory. $f_t = \sigma(\tilde{f}_t)$ or $\exp(\tilde{f}_t)$, $\tilde{f}_t = w_f^\top x_t + r_f h_{t-1} + b_f$,
 $o_t = \sigma(\tilde{o}_t)$, $\tilde{o}_t = w_o^\top x_t + r_o h_{t-1} + b_o$
3. **sLSTM cell update (scalar memory):** exponential gates write/decay a scalar state. $c_t = f_t c_{t-1} + i_t z_t$,
 $z_t = \phi(\tilde{z}_t)$, $\tilde{z}_t = w_z^\top x_t + r_z h_{t-1} + b_z$
4. **sLSTM normalization & output:** a running “strength” normalizer keeps the ratio stable. $n_t = f_t n_{t-1} + i_t$,
 $h_t = o_t \frac{c_t}{n_t}$
5. **mLSTM covariance (matrix memory) update:** stores key–value pairs via an outer product. $C_t = f_t C_{t-1} + i_t v_t k_t^\top$,
 $q_t = W_q x_t + b_q$,
 $k_t = 1/\sqrt{d} W_k x_t + b_k$,
 $v_t = W_v x_t + b_v$
6. **mLSTM retrieval with stabilizer:** read by multiplying the memory with a query and dividing by a gate-strength normalizer. $n_t = f_t n_{t-1} + i_t k_t$,
 $h_t = o_t \odot \frac{C_t q_t}{\max(|n_t^\top q_t|, 1)}$

Where $x_t \in \mathbb{R}^{d_x}$ denotes the input vector at time step t (e.g., the embedding of the current item), and $h_{t-1} \in \mathbb{R}^d$ is the previous hidden state vector. The scalars $i_t, f_t, o_t \in \mathbb{R}_+$ represent the input, forget, and output gates, respectively, with $\tilde{i}_t, \tilde{f}_t, \tilde{o}_t$ their pre-activation values. The parameters $w_* \in \mathbb{R}^{d_x}$ and $r_* \in \mathbb{R}^d$ are input-to-hidden and hidden-to-hidden weight vectors, and $b_* \in \mathbb{R}$ are scalar bias terms. The scalar cell state $c_t \in \mathbb{R}$ stores accumulated memory, while $n_t \in \mathbb{R}_+$ is a scalar normalizer that stabilizes exponential growth. The function $\sigma(\cdot)$ denotes the logistic sigmoid, $\exp(\cdot)$ the exponential gate, and $\phi(\cdot)$ a bounded nonlinearity Beck et al. [2024].

For the matrix-memory variant, $C_t \in \mathbb{R}^{d \times d}$ is the covariance (key–value) memory matrix, with $q_t, k_t, v_t \in \mathbb{R}^d$ denoting query, key, and value vectors computed from the input via learned matrices $W_q, W_k, W_v \in \mathbb{R}^{d \times d_x}$ and biases b_q, b_k, b_v . The operator $\odot$ denotes element-wise multiplication.

The resulting hidden state $h_t \in \mathbb{R}^d$ is used as the representation for downstream prediction, such as next-item ranking Beck et al. [2024].

xLSTM employs the same next-item prediction loss as LSTM and Transformer based models, the architecture differs only in how sequential representations are computed Beck et al. [2024]. Below is the Softmax Cross-Entropy Loss function.

$$\begin{aligned} s_{t+1} &= E^\top h_t, \\ p(y_{t+1} | h_t) &= \text{softmax}(s_{t+1}), \\ \mathcal{L} &= - \sum_{t=1}^T \log p(y_{t+1}^* | h_t), \end{aligned}$$

Where $h_t \in \mathbb{R}^d$ is the hidden state vector produced by the sequence model at time step t , and $E \in \mathbb{R}^{N \times d}$ denotes the shared item embedding (output) matrix, where N is the total number of items in the catalog and each row corresponds to an item representation. The score vector $s_{t+1} \in \mathbb{R}^N$ contains unnormalized scores for all items at the next time step Beck et al. [2024]. The conditional distribution $p(y_{t+1} | h_t) \in \mathbb{R}^N$ is obtained by applying the softmax function, producing probabilities over item IDs. The scalar $y_{t+1}^* \in \{1, \dots, N\}$ denotes the ground-truth next item index, and $\mathcal{L}$ is the softmax cross-entropy loss summed over all time steps T . By minimizing this loss over all users and sequences, the model learns to rank the correct next item higher than other candidates in next-item recommendation tasks such as MovieLens.

These architectural improvements directly target the accuracy–efficiency trade-offs in RQ3. The structured memory and gating behavior serve as the basis for embedding saturation and geometric analysis in RQ4.

6 Model Architectures Compared

SAS4RecKang and McAuley [2018], BERT4RecSun et al. [2019], and xLSTM Beck et al. [2024] differ fundamentally in how they transform an input sequence of item embeddings $x_t \in \mathbb{R}^d$ into hidden representations for next-item prediction. SASRec computes context-aware representations using causal self-attention, where queries Q , keys K , and values V are linear projections of the embedding sequence E_b , and a causal mask M ensures that each position attends only to past items. BERT4Rec similarly relies on self-attention but operates on bidirectionally encoded hidden states $H \in \mathbb{R}^{L \times d}$, optimizing a Cloze-style objective that predicts masked items using final-layer representations $h_t^{(L)}$ and a shared item embedding matrix $E \in \mathbb{R}^{N \times d}$ Beck et al. [2024].

In contrast, xLSTM Beck et al. [2024] maintains recurrent hidden states $h_t \in \mathbb{R}^d$ and explicit memory states (c_t for sLSTM and C_t for mLSTM), which are updated through gated additive and multiplicative operations without computing pairwise item interactions. While SASRec and BERT4Rec incur quadratic complexity due to the $QK^\top$ attention operation Beck et al. [2024], xLSTM updates its memory linearly with sequence length. Despite these architectural differences, all three models compute item scores $s_{t+1} = E^\top h_t$ and learn parameters by minimizing a softmax- or sigmoid-based loss over the ground-truth next item y_{t+1}^* , enabling ranking over the full item catalog Beck et al. [2024].

We evaluate and compare the following sequential recommender architectures, each representing a distinct modeling paradigm for user–item sequences $\{x_1, \dots, x_T\}$, where $x_t \in \mathbb{R}^d$ denotes the embedding of the item interacted with at time t .

xLSTM Beck et al. [2024] is the proposed architecture in this work, integrating scalar and matrix-based memory states ($c_t \in \mathbb{R}^d$, $C_t \in \mathbb{R}^{d \times d}$), chunkwise attention, and bidirectional memory routing Kang and McAuley [2018]. This enables efficient modeling of long-range temporal dependencies while maintaining linear complexity in sequence length and supporting GPU-optimized kernels for stable and scalable training Kang and McAuley [2018].

SAS4Rec Kang and McAuley [2018] is a Transformer-based sequential recommender that replaces recurrence with causal self-attention. It computes context-aware representations using query, key, and value projections (Q, K, V) over past items, ensuring that each position attends only to preceding interactions Kang and McAuley [2018]. This design captures long-range dependencies but incurs with the quadratic complexity with respect to the sequence length T Kang and McAuley [2018].

BERT4Rec Vaswani et al. [2017] adopts a bidirectional Transformer encoder trained with a masked-item prediction objective Vaswani et al. [2017] Sun et al. [2019]. By allowing each hidden representation $\mathbf{h}_t$ to attend to both past and future items during training, it learns item-item dependencies, at the cost of higher computational and memory overhead compared to unidirectional models.

6.1 Evaluation Metrics

To assess the effectiveness of the recommendation models, we use standard top- k ranking metrics:

- **Recall@10**: Measures the proportion of times the correct next item appears in the top-10 predicted list. It captures model coverage and hit rate.
- **MRR@10 (Mean Reciprocal Rank)**: Evaluates the average of reciprocal ranks of the first relevant item. It reflects how high the correct item is ranked in the list.
- **NDCG@10 (Normalized Discounted Cumulative Gain)**: Considers both the relevance and position of items in the top-10 list, rewarding higher placements of correct predictions.

Recall@K

Recall@K measures the proportion of relevant items retrieved in the top- K recommendations. It is defined as:

$$Recall@K = \frac{|\{relevantitems\} \cap \{top-Kpredicteditems\}|}{|\{relevantitems\}|} \quad (1)$$

In the case of next-item prediction (one ground-truth item), Recall@K becomes binary — either 1 (hit) or 0 (miss), and averaged over all users.

Mean Reciprocal Rank (MRR@K)

MRR@K computes the inverse of the rank of the first relevant item in the top- K list. The mean is taken over all users:

$$MRR@K = \frac{1}{|U|} \sum_{u=1}^{|U|} \frac{1}{rank_u} \quad (2)$$

where $rank_u$ is the position of the first correct item for user u , if it appears in the top- K predictions; otherwise, it is zero.

Normalized Discounted Cumulative Gain (NDCG@K)

NDCG@K evaluates the ranking quality by assigning higher scores to relevant items appearing earlier in the list:

$$NDCG@K = \frac{1}{|U|} \sum_{u=1}^{|U|} \frac{1}{IDCG_u@K} \sum_{i=1}^K \frac{\mathbb{I}[item_i is relevant]}{\log_2(i+1)} \quad (3)$$

where $IDCG_u@K$ is the ideal DCG (maximum possible DCG for user u) and $\mathbb{I}[\cdot]$ is the indicator function.

Other Factors/Metrics

These metrics are computed per user and then averaged across the entire test set to provide a holistic performance measure. Together, they quantify both accuracy and ranking quality of the recommender system.

Offline / Online / User Studies: Offline—evaluate next-item prediction on time-ordered logs; Online—A/B test live sessions (CTR, dwell, retention); User studies—human judgments of sequence relevance, diversity, and explanations.

Task-centric categories: *Retrieval* ranks a large candidate pool for the next item; *Classification* predicts sequence labels (e.g., click/skip, genre); *Regression* predicts real-valued outcomes (rating, dwell time, return probability).

Beyond-accuracy metrics: Assess list quality beyond Recall/MRR—focusing on diversity, novelty, coverage, serendipity, and explainability.

- **Diversity:** Ensure the recommended next-item list spans varied facets (e.g., artists/genres) rather than near-duplicates of recent history.
- **Intra-list diversity (ILD):** Mean pairwise dissimilarity within the recommended list (e.g., $1 - \text{sim}(\text{embedding}_i, \text{embedding}_j)$).
- **Entropy:** Shannon entropy of category/item distribution in the list (or over exposures) to quantify variety.
- **Novelty (global / individual):** *Global*—favor long-tail items rare in logs; *Individual*—favor items new to the user given their sequence history.
- **Coverage:** Fraction of the catalog (and/or users) the model can and does recommend to over time.
- **Serendipity:** Items that are relevant to the sequence yet unexpectedly different from the user’s usual consumption.
- **Explainability:** User-facing reasons tied to sequence evidence (e.g., attention to specific past items, co-occurrence paths)

6.2 Training Pipeline and Hyperparameters

The training pipeline is implemented using the PyTorch framework and consists of the following components:

- **Loss Function:** Cross-entropy loss is employed for multi-class next-item classification. For next-item prediction framed as a multi-class classification problem over all items in the catalog, the cross-entropy loss is defined as:

$$\mathcal{L}_{CE} = - \sum_{i=1}^N \log \left(\frac{e^{z_{i,y_i}}}{\sum_{j=1}^C e^{z_{i,j}}} \right) \quad (4)$$

where:

- N is the number of training examples (users or sequences),
- C is the total number of candidate items (classes),
- $z_{i,j}$ is the logit score (pre-softmax output) for class j for example i ,
- y_i is the true class (next item index) for example i .

Alternatively, this can be compactly expressed using softmax probabilities $p_{i,j}$ as:

$$\mathcal{L}_{CE} = - \sum_{i=1}^N \log(p_{i,y_i}) \quad \text{where} \quad p_{i,j} = \frac{e^{z_{i,j}}}{\sum_{k=1}^C e^{z_{i,k}}} \quad (5)$$

- **Optimizer:** All models were trained using the Adam stochastic gradient-based optimization algorithm `kingma2015adam`. A fixed initial learning rate of 1×10^{-3} was employed consistently across all five datasets and three model architectures. This uniform optimization setting was chosen to ensure stable convergence behavior and to facilitate fair and controlled comparisons among different model variants, independent of dataset scale or architectural complexity.

- **Learning Rate Control:** The learning rate remained constant throughout the optimization process, with convergence regulation achieved exclusively through validation-based early stopping. This design choice simplifies the training pipeline and ensures that observed performance improvements are attributable to the learning dynamics of the models srivastava2014dropout.
- **Batching and Sequence Padding:** User interaction sequences were converted into fixed-length representations by left-padding shorter sequences with zero-valued tokens up to a predefined maximum sequence length (MAX_LEN). This padding strategy enables the construction of uniform tensor shapes required for efficient batch-wise processing. Mini-batches were generated using PyTorch’s data loading utilities, allowing for parallelized computation and effective utilization of GPU hardware during training quadrana2018sequenceaware.
- **Training Duration:** Each model was trained for a predefined maximum number of epochs. To mitigate overfitting and reduce unnecessary computational overhead, an early stopping criterion (patience = 3) was employed based on validation performance srivastava2014dropout. Specifically, training was terminated when Recall@10 on the validation set failed to improve for three consecutive epochs. This strategy ensures that training concludes once performance saturates, thereby promoting models with stronger generalization capability.

Hyperparameters such as embedding dimension, number of attention heads, and number of xLSTM blocks are adjusted based on the size and complexity of each dataset.

Table 3: xLSTM and DataLoader hyperparameters per dataset.

Dataset	Embed Dim	Heads	Blocks	Batch Size	Workers
100K	64	2	1	128	2
1M	128	2	2	128	2
10M	128	2	2	1024	2
Steam	256	8	4	1024	2

6.3 Cold Start and Sparsity Handling

Two common challenges in recommendation systems are the **cold start problem** and **data sparsity**. In this work:

- **Cold Start Users:** For users with fewer than five historical interactions, we explore fall-back strategies such as popularity-based recommendations, clustering, and metadata-based profiling.
- **Sparsity:** Dataset sparsity is quantified (often exceeding 98%), and addressed via negative sampling and emphasizing recent interactions to capture temporal relevance.
- **Evaluation Strategy:** Cold-start users are excluded from the main metric computation but separately analyzed for robustness.

Cold Start Problem: Some of the commonly used approaches were: 1. Clustering Approach, 2. Profile Based (Meta Data) Approach, 3. Hierarchical approach, and 4. Novelty or Randomness Approach.

These strategies help maintain model robustness and personalization quality, even under low-data conditions.

7 Experimental Results and Interpretation

7.1 Comparative Performance Results and Analysis

7.2 Results in Response to Research Questions

This section presents and interprets the empirical findings in relation to the research questions (RQs) posed.

Table 4: Performance comparison across models, datasets, and sequence lengths (excluding training time, LR, and hit rate).

Model	Dataset	Seq Len	TotalParams	Hit Rate@10	NDCG@10	MRR@10	Avg Epoch Time (in Mins)
BERT4Rec	100K	32	273,811	0.1273 $\pm$ 0.0084	0.0620 $\pm$ 0.0044	0.0425 $\pm$ 0.0042	7.0
		64	275,859	0.1217 $\pm$ 0.0091	0.0582 $\pm$ 0.0056	0.0392 $\pm$ 0.0050	7.1
		128	279,955	0.1202 $\pm$ 0.0097	0.0584 $\pm$ 0.0042	0.0399 $\pm$ 0.0035	7.2
	1M	32	1,370,875	0.2766 $\pm$ 0.0149	0.1573 $\pm$ 0.0103	0.1209 $\pm$ 0.0088	95.4
		64	1,374,971	0.2752 $\pm$ 0.0155	0.1550 $\pm$ 0.0106	0.1184 $\pm$ 0.0090	95.2
		128	1,383,163	0.2777 $\pm$ 0.0174	0.1568 $\pm$ 0.0110	0.1200 $\pm$ 0.0092	101.3
	10M	32	8,712,886	0.3112 $\pm$ 0.0000	0.1494 $\pm$ 0.0000	0.1874 $\pm$ 0.0000	1,347.8
		64	8,721,078	0.2744 $\pm$ 0.0000	0.1232 $\pm$ 0.0000	0.1585 $\pm$ 0.0000	1,072.7
		128	8,737,462	0.3171 $\pm$ 0.0000	0.1513 $\pm$ 0.0000	0.1902 $\pm$ 0.0000	2,129.6
	Musik4All	32	13,600,220	0.3565 $\pm$ 0.0000	0.2626 $\pm$ 0.0000	0.2332 $\pm$ 0.0000	236.0
		64	13,604,316	0.3653 $\pm$ 0.0000	0.2668 $\pm$ 0.0000	0.2362 $\pm$ 0.0000	245.8
		128	13,612,508	0.3644 $\pm$ 0.0000	0.2720 $\pm$ 0.0000	0.2431 $\pm$ 0.0000	276.9
	Amazon Software	32	5,985,824	0.1020 $\pm$ 0.0000	0.0732 $\pm$ 0.0000	0.0643 $\pm$ 0.0000	1.5
		64	5,989,920	0.1008 $\pm$ 0.0000	0.0712 $\pm$ 0.0000	0.0620 $\pm$ 0.0000	1.5
		128	5,998,112	0.0985 $\pm$ 0.0000	0.0692 $\pm$ 0.0000	0.0601 $\pm$ 0.0000	1.7
SAS4Rec	100K	32	269,139	0.1278 $\pm$ 0.0098	0.0623 $\pm$ 0.0038	0.0427 $\pm$ 0.0028	5.9
		64	271,187	0.1247 $\pm$ 0.0106	0.0606 $\pm$ 0.0059	0.0414 $\pm$ 0.0051	6.0
		128	275,283	0.1283 $\pm$ 0.0106	0.0615 $\pm$ 0.0056	0.0415 $\pm$ 0.0044	6.0
	1M	32	1,353,339	0.2109 $\pm$ 0.0120	0.1191 $\pm$ 0.0083	0.0912 $\pm$ 0.0072	82.2
		64	1,357,435	0.1427 $\pm$ 0.0072	0.0798 $\pm$ 0.0053	0.0606 $\pm$ 0.0047	82.3
		128	1,365,627	0.0835 $\pm$ 0.0048	0.0461 $\pm$ 0.0032	0.0347 $\pm$ 0.0027	89.9
	10M	32	8,645,046	0.2143 $\pm$ 0.0000	0.1246 $\pm$ 0.0000	0.0972 $\pm$ 0.0000	674.7
		64	8,653,238	0.1271 $\pm$ 0.0000	0.0713 $\pm$ 0.0000	0.0544 $\pm$ 0.0000	1,170.4
		128	8,669,622	0.0727 $\pm$ 0.0000	0.0400 $\pm$ 0.0000	0.0301 $\pm$ 0.0000	2,372.6
	Musik4All	32	13,582,427	0.3280 $\pm$ 0.0000	0.2397 $\pm$ 0.0000	0.2122 $\pm$ 0.0000	232.9
		64	13,586,523	0.3087 $\pm$ 0.0000	0.2285 $\pm$ 0.0000	0.2036 $\pm$ 0.0000	254.3
		128	13,594,715	0.2815 $\pm$ 0.0000	0.2107 $\pm$ 0.0000	0.1886 $\pm$ 0.0000	242.9
	Amazon Software	32	5,968,288	0.0050 $\pm$ 0.0000	0.0019 $\pm$ 0.0000	0.0009 $\pm$ 0.0000	1.3
		64	5,972,384	0.0050 $\pm$ 0.0000	0.0019 $\pm$ 0.0000	0.0009 $\pm$ 0.0000	1.3
		128	5,980,576	0.0050 $\pm$ 0.0000	0.0019 $\pm$ 0.0000	0.0009 $\pm$ 0.0000	1.7
xLSTM	100K	32	269,188	0.1041 $\pm$ 0.0070	0.0509 $\pm$ 0.0036	0.0350 $\pm$ 0.0029	9.6
		64	269,188	0.1057 $\pm$ 0.0083	0.0522 $\pm$ 0.0050	0.0361 $\pm$ 0.0046	10.0
		128	269,188	0.1036 $\pm$ 0.0095	0.0511 $\pm$ 0.0044	0.0355 $\pm$ 0.0038	10.3
	1M	32	1,376,904	0.2625 $\pm$ 0.0146	0.1492 $\pm$ 0.0102	0.1148 $\pm$ 0.0089	143.1
		64	1,376,904	0.2603 $\pm$ 0.0151	0.1483 $\pm$ 0.0104	0.1143 $\pm$ 0.0089	147.0
		128	1,376,904	0.2605 $\pm$ 0.0155	0.1479 $\pm$ 0.0107	0.1136 $\pm$ 0.0092	164.2
	10M	32	8,698,176	0.3138 $\pm$ 0.0000	0.1910 $\pm$ 0.0000	0.1533 $\pm$ 0.0000	1,218.7
		64	8,698,176	0.3216 $\pm$ 0.0000	0.1962 $\pm$ 0.0000	0.1577 $\pm$ 0.0000	2,208.3
		128	8,698,176	0.3183 $\pm$ 0.0000	0.1937 $\pm$ 0.0000	0.1555 $\pm$ 0.0000	4,239.2
	Musik4All	32	13,558,664	0.3496 $\pm$ 0.0000	0.2591 $\pm$ 0.0000	0.2309 $\pm$ 0.0000	701.5
		64	13,558,664	0.3439 $\pm$ 0.0000	0.2568 $\pm$ 0.0000	0.2296 $\pm$ 0.0000	1101.2
		128	13,558,664	0.3505 $\pm$ 0.0000	0.2605 $\pm$ 0.0000	0.2324 $\pm$ 0.0000	1851.4
	Amazon Software	32	5,973,896	0.0829 $\pm$ 0.0000	0.0600 $\pm$ 0.0000	0.0528 $\pm$ 0.0000	4.1
		64	5,973,896	0.0842 $\pm$ 0.0000	0.0598 $\pm$ 0.0000	0.0523 $\pm$ 0.0000	6.7
		128	5,973,896	0.0846 $\pm$ 0.0000	0.0619 $\pm$ 0.0000	0.0548 $\pm$ 0.0000	12.1

RQ1 - Performance Scaling (Table 4) : Across the MovieLens 100K, 1M, and 10M datasets, xLSTM demonstrates a clear positive scaling trend. While performance on the smallest dataset (ML 100K) lags behind the Transformer models, xLSTM significantly improves as the interaction histories emerge. xLSTM matches BERT4Rec’s Recall@10 (26–27%) on the MovieLens 1M dataset, indicating that it scales effectively with larger interaction histories. As dataset size increases, the performance gap between models diminishes, suggesting that xLSTM converges toward BERT4Rec’s performance under rich-data conditions.

RQ2 - Sequence Sensitivity (Table 4) : Experiments across sequence lengths of 32, 64, and 128 show that xLSTM exhibits increasing performance variance as sequences grow longer, reflecting its sensitivity to temporal window size. The performance increases with longer user interaction sequences. This highlights xLSTM’s sensitivity to sequence length, which could impact its robustness across varying user histories. (Table 4)

RQ3 - Trade-offs and Efficiency (Table 4): While xLSTM achieves competitive accuracy on large-scale datasets, it incurs greater computational cost, especially in smaller-scale settings where lightweight models like BERT4Rec can deliver similar performance with lower resource overhead. (Table 4)

RQ4 — Embedding Utilization, Saturation, and Representational Diversity:

Embedding geometry analyses reveal that xLSTM makes substantially more effective use of embedding space than Transformer baselines. While BERT4Rec and SAS4Rec exhibits the anisotropic embedding structures driven by the popularity bias, xLSTM produces nearly isotropic embeddings with lower hubness, higher intrinsic dimensionality, and more uniform variance distribution. CKA

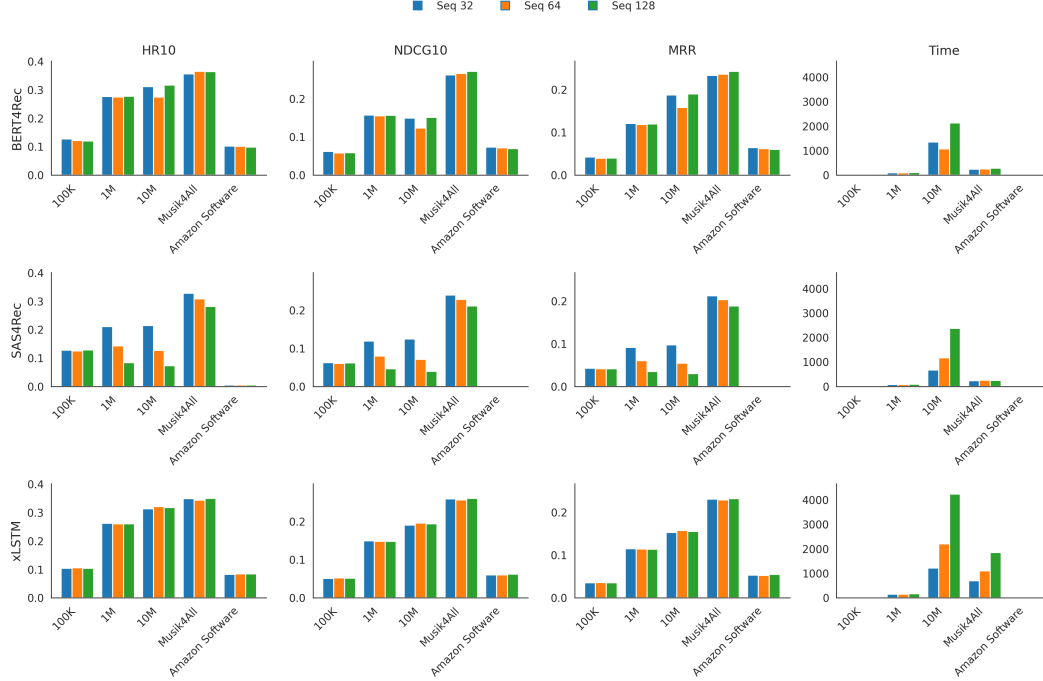


Figure 5: Comparison Results

similarity studies further show that the xLSTM learns fundamentally different, sequence oriented embedding structures rather than compressing items along global similarity axes.

These findings illustrate that while xLSTM is a promising architecture, its performance gains come with trade-offs in computational efficiency and sequence sensitivity, which must be considered in deployment scenarios.

7.3 Scaling Characteristics and Computational Trade-offs

Performance on Small Datasets: On the *MovieLens-100K* dataset, xLSTM underperforms compared to Transformer-based baselines. It achieves a **HitRate@10 of approximately 10.5%**, whereas SASRec reaches around **12.8%**.

Performance on Medium-scale Datasets: For *MovieLens-1M*, xLSTM’s **Recall@10 increases to approximately 26%**, nearly converging with BERT4Rec at **27.7%**, owing to xLSTM’s improved gating and memory mechanisms.

Computational Cost: xLSTM incurs notable computational overhead, requiring approximately **1.5x–2x longer training time per epoch** than LSTM-based baselines.

Temporal Complexity: Like standard LSTM, xLSTM retains a linear temporal complexity of $\mathcal{O}(n)$. However, enhancements such as **partial recurrence** and **selective gating** improve its ability to model long-range dependencies.

Model Complexity: The parameter count in xLSTM scales with $\mathcal{O}(k)$, where k denotes the memory depth. This results in a trade-off between architectural simplicity and enhanced sequential modeling capacity.

7.4 Quantitative Evaluation (Recall@10, MRR@10, NDCG@10)

This subsection presents the core performance metrics used to evaluate the recommendation accuracy of different models. Recall@10 measures how often the correct item is among the top-10 predicted, MRR@10 evaluates the rank of the first correct item, and NDCG@10 captures both correctness and ranking position. The xLSTM model showed a strong baseline with Recall@10 0.29, indicating

Actual Sequence:	
User ID: 3	
Input sequence:	
- Item 386: L.A. Confidential (1997) - Item 302: Titanic (1997) - Item 127: George of the Jungle (1997) - Item 349: Wag the Dog (1997) - Item 540: I Know What You Did Last Summer (1997) - Item 171: Full Monty, The (1997) - Item 265: Kolya (1996) - Item 250: Contact (1997) - Item 347: Mother (1996) - Item 346: Cop Land (1997) - Item 273: Ice Storm, The (1997) - Item 321: Everyone Says I Love You (1996) - Item 309: Fly Away Home (1996) - Item 298: Liar Liar (1997) - Item 716: Saint, The (1997) - Item 1003: Until the End of the World (Bis ans Ende der Welt) (1991) - Item 39: Independence Day (ID4) (1996) - Item 252: My Best Friend's Wedding (1997) - Item 199: Weekend at Bernie's (1989) - Item 673: City Slickers II: The Legend of Curly's Gold (1994) - Item 17: Star Wars (1977) - Item 305: Secrets & Lies (1996)	
Predicted Next Sequence	
True next item: Item 244: Dead Man Walking (1995) (logit: 4.8375, confidence: 0.0215)	
Top-10 Predictions with scores:	
1. Item 242: Fargo (1996) (logit: 5.5718, confidence: 0.0448) 2. Item 286: Sense and Sensibility (1995) (logit: 5.0981, confidence: 0.0279) 3. Item 14: Chasing Amy (1997) (logit: 5.0799, confidence: 0.0274) 4. Item 220: Cold Comfort Farm (1995) (logit: 5.0262, confidence: 0.0259) 5. Item 303: Emma (1996) (logit: 4.9382, confidence: 0.0238) 6. Item 464: People vs. Larry Flynt, The (1996) (logit: 4.9179, confidence: 0.0233) 7. Item 244: Dead Man Walking (1995) (logit: 4.8375, confidence: 0.0215) 8. Item 300: Time to Kill, A (1996) (logit: 4.7955, confidence: 0.0206) 9. Item 36: Jerry Maguire (1996) (logit: 4.7657, confidence: 0.0200) 10. Item 252: My Best Friend's Wedding (1997) (logit: 4.6183, confidence: 0.0173)	

Figure 6: Sample Predictions

nearly 3 out of 10 correct predictions. These metrics are computed over test datasets and tracked across training epochs.

7.5 Qualitative Insights and Sample Predictions

Here, we analyze model behavior by reviewing specific prediction outputs. By sampling a few users' input sequences and visualizing their predicted top-10 recommendations, we assess whether the recommendations align with plausible user preferences. These examples help interpret how well the model captures temporal dynamics, genre preferences, or recency effects, offering explainability beyond numeric metrics.

7.6 Embedding Geometry Analysis for Sequential Recommenders Models

Using the saved best-performing models, we conducted a series of embedding geometry analyses on three architectures — BERT4Rec, SASRec, and xLSTM — primarily utilizing the larger MovieLens-10M models for the following studies.

A. L2 Norm : First, the L2 norm measures the overall magnitude (length) of a vector as the square root of the sum of its squared components. It tells us how strong or energetic a vector is — in embeddings, it reflects how much information or weight each item's representation carries in the model's space.

In our embeddings, all three embedding matrices share the same shape.

$$E_{BERT4Rec}, E_{SASRec}, E_{xLSTM} \in \mathbb{R}^{10678 \times 257}$$

where $N = 10678$ is the number of movie items and $D = 256$ is the embedding dimension.

$$\|E_i\|_2 = \sqrt{\sum_{j=1}^D e_{ij}^2}$$

$$Mean = 1N \sum_i \|E_i\|_2, \quad Std = \sqrt{1N \sum_i (\|E_i\|_2 - Mean)^2}, \quad Range = [\min_i \|E_i\|_2, \max_i \|E_i\|_2]$$

$$p(\|E_i\|_2) = HistogramDensity(\|E_i\|_2), \quad NormStats = \{Mean, Std, Min, Max\}$$

float

B. Mean and variance per embedding dimension: The mean and variance per embedding dimension measure how values are distributed along each axis of the embedding space. They show whether some dimensions dominate (high variance) or stay near zero (low variance). This helps

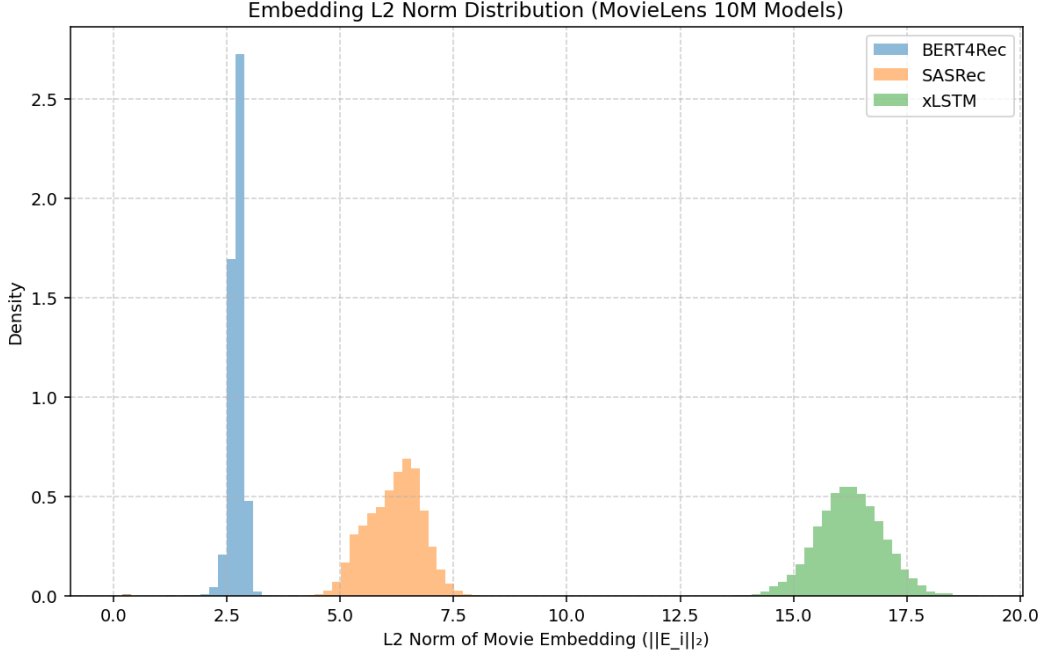


Figure 7: Enter Caption

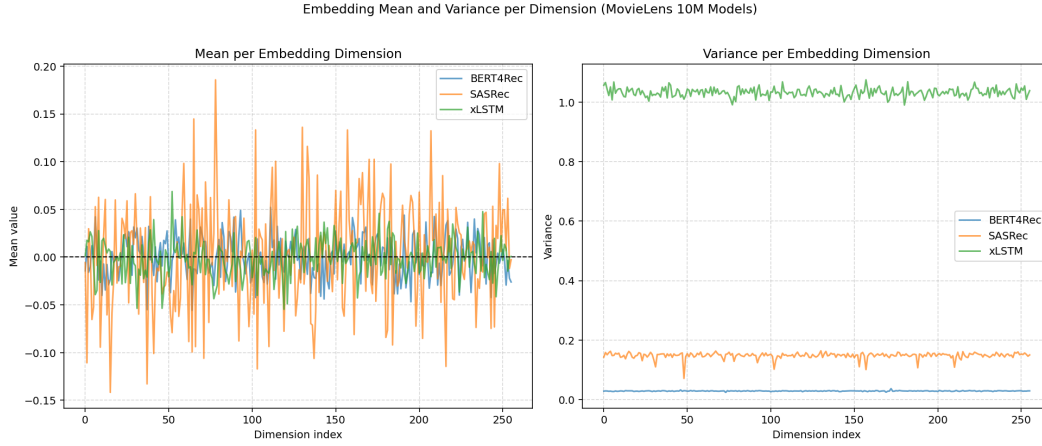


Figure 8: Enter Caption

detect anisotropy — uneven use of embedding dimensions — which affects how well the model encodes diverse item relationships. A crucial study for understanding anisotropy (uneven information spread across dimensions).

$$E \in \mathbb{R}^{N \times D}, \quad \mu_j = \frac{1}{N} \sum_{i=1}^N e_{ij}, \quad \sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (e_{ij} - \mu_j)^2$$

$$Mean(abs) = \frac{1}{D} \sum_{j=1}^D |\mu_j|, \quad Var(mean) = \frac{1}{D} \sum_{j=1}^D (\mu_j - \bar{\mu})^2, \quad Var(range) = [\min_j \sigma_j^2, \max_j \sigma_j^2]$$

Each embedding dimension j has a mean μ_j and variance σ_j^2 computed across all items, describing how that axis behaves overall.

By comparing their averages and ranges, we can detect *anisotropy* — whether certain dimensions dominate while others remain near zero.

C. Embedding Spectrum Analysis: Analyzing the eigenvalue spectrum of the embedding covariance matrix shows how information is distributed across embedding directions, directly addressing RQ4 by revealing each model’s intrinsic dimensionality and capacity utilization. Embedding spectrum analysis evaluates how much variance each principal component captures. A steep decay indicates that most information lies in only a few directions, while a flatter spectrum suggests, more distributed representation et al. [2018, 2019a,b].

BERT4Rec and SAS4Rec exhibit steep eigenvalue decay, meaning a small number of dominant directions shape the representation space. This results in an anisotropic embedding geometry where information is concentrated in a few axes. xLSTM’s spectrum decays more gradually, indicating that variance is spread across many dimensions. This reflects higher intrinsic dimensionality and more effective use of embedding capacity. A flatter tail means the model preserves more independent features instead of collapsing them into redundant directions. In Transformer models, sharp decay often correlates with popularity-driven compression, where common items dominate the representation space. Therefore, xLSTM encodes items in a more uniform and diverse manner, supporting latent structure and reducing over-reliance on a few principal components.

(Figure 5.5 Row 1: Eigenvalue decay curves “spectra”, for BERT4Rec, SAS4Rec, and xLSTM, showing how variance is distributed across principal embedding directions)

D. Variance Distribution and Intrinsic Dimension Study: This analysis examines how many principal components are required to explain a given proportion of total variance, providing an estimate of each model’s effective intrinsic dimensionality. Models that achieve high explained variance using only a few components tend to compress information into a narrow subspace, whereas models requiring many components distribute information more broadly across the embedding space et al. [2018, 2019a,b].

BERT4Rec and SAS4Rec reach approximately 90 percent explained variance within the first 50 components, indicating strong redundancy and a highly compressed latent structure. xLSTM requires nearly 200 components to reach the same threshold, suggesting much more distributed and expressive representation et al. [2018, 2019a,b]. A more gradual cumulative-variance slope reflects broader feature usage and reduced risk of rank collapse. These observations align with the intrinsic dimension estimates (180 / 204 / 250), reinforcing the differences in representational capacity. Overall, xLSTM demonstrates the highest effective dimensionality, while BERT4Rec and SAS4Rec rely on more compact—and more redundant—embedding spaces et al. [2018, 2019a,b].

(Figure 5.5 Row 2: Cumulative explained variance plots, indicating how quickly each model concentrates information into leading components.)

E. Hubness and Popularity Bias Evaluation This analysis measures how frequently each item appears in other items’ top-10 nearest neighbors in cosine space. High-dimensional embeddings often suffer from hubness, where a small number of items become disproportionately common neighbors, typically reflecting popularity bias or collapsed representations Radovanović et al. [2010], et al. [2015, 2017].

BERT4Rec and SAS4Rec show highly right-skewed hubness distributions, where a small set of items appear hundreds of times as nearest neighbors. This indicates that similarity space is dominated by a few “hub” items, reducing diversity and amplifying popularity bias Radovanović et al. [2010], et al. [2015, 2017]. xLSTM exhibits a much more symmetric and narrow distribution, where most items appear within a similar frequency range. The lower hubness score (Gini 0.18) demonstrates improved fairness and stronger long-tail representation. Transformer-based embeddings appear prone to overemphasizing frequent items, likely due to attention mechanisms that correlate strongly with item popularity. In contrast, xLSTM produces a flatter and more balanced similarity graph, enhancing recommendation diversity while mitigating popularity-driven collapse.

(Figure 5.5 Row 3: Hubness histograms, k=10, illustrating how often items appear as nearest neighbors, where higher skew indicates stronger hubness.)

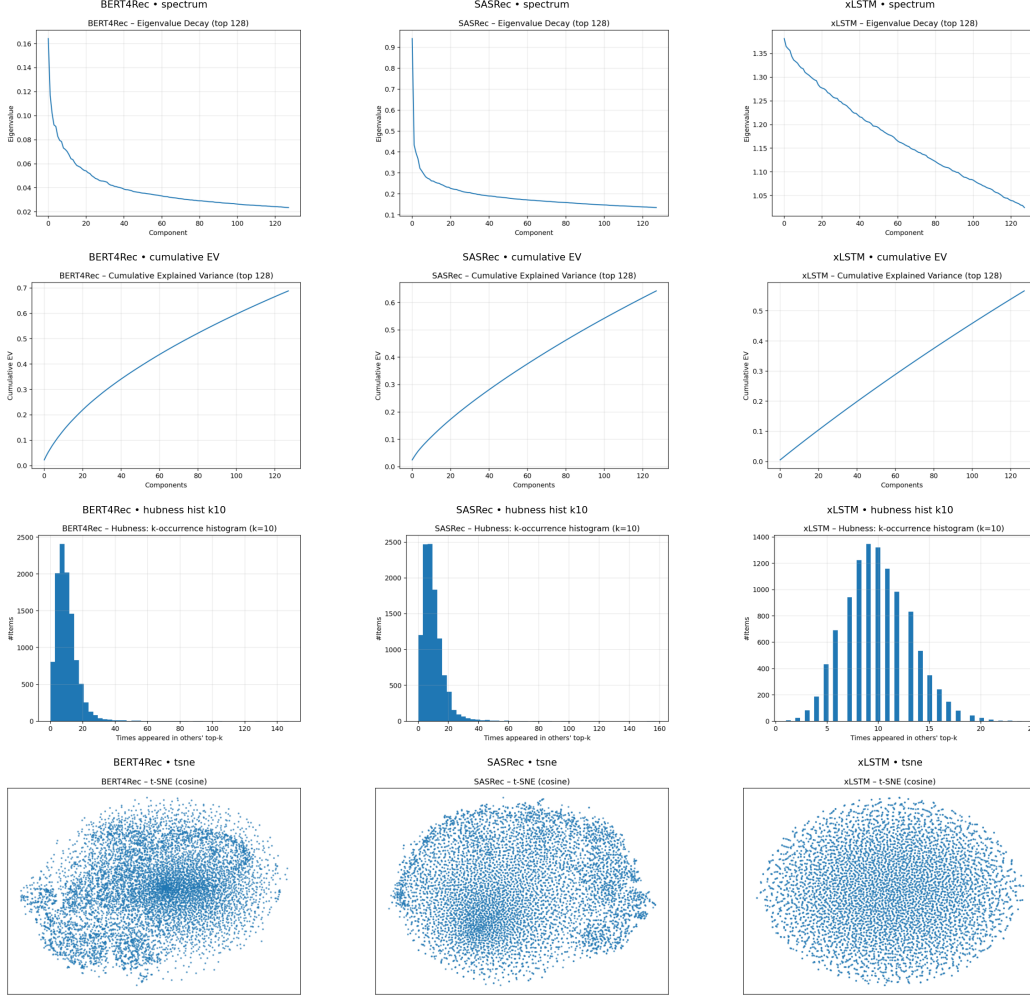


Figure 9: Embedding Analysis

F. t-SNE Embedding Space Visualization To qualitatively examine the structure of each model’s embedding space, we project the 256-dimensional item embeddings into two dimensions using t-SNE. This nonlinear projection attempts to preserve local neighborhoods, allowing us to visually assess clustering behavior, spatial density, and overall geometric structure van der Maaten and Hinton [2008].

BERT4Rec and SAS4Rec form dense, centrally concentrated clusters, producing elliptical, blob-like structures Arora et al. [2016], Ethayarajh [2019], Li et al. [2020]. This reflects anisotropy in the embedding space and aligns with earlier evidence of hubness and directional collapse. xLSTM shows a more diffuse and ring-like distribution, where points occupy the space more evenly. This suggests greater isotropy and a more balanced similarity structure. The reduced presence of tight clusters indicates less genre- or popularity-driven collapse, with features distributed more smoothly. Visually, xLSTM’s embedding space appears broader and more uniformly populated, reinforcing the idea of higher representational diversity. Such geometry supports more stable nearest-neighbor retrieval and better generalization across different item types.

(Figure 5.5, Row 4 displays the resulting t-SNE projections.)

Overall, Transformers (BERT4Rec, SASRec) learn narrow, popularity-biased manifolds; xLSTM learns a broad, isotropic embedding landscape — fairer, and geometrically independent.

G. Embedding anisotropy visualization: The Anisotropy Index (AI) measures how uniformly embeddings are distributed in space — it’s the mean cosine similarity between random pairs of

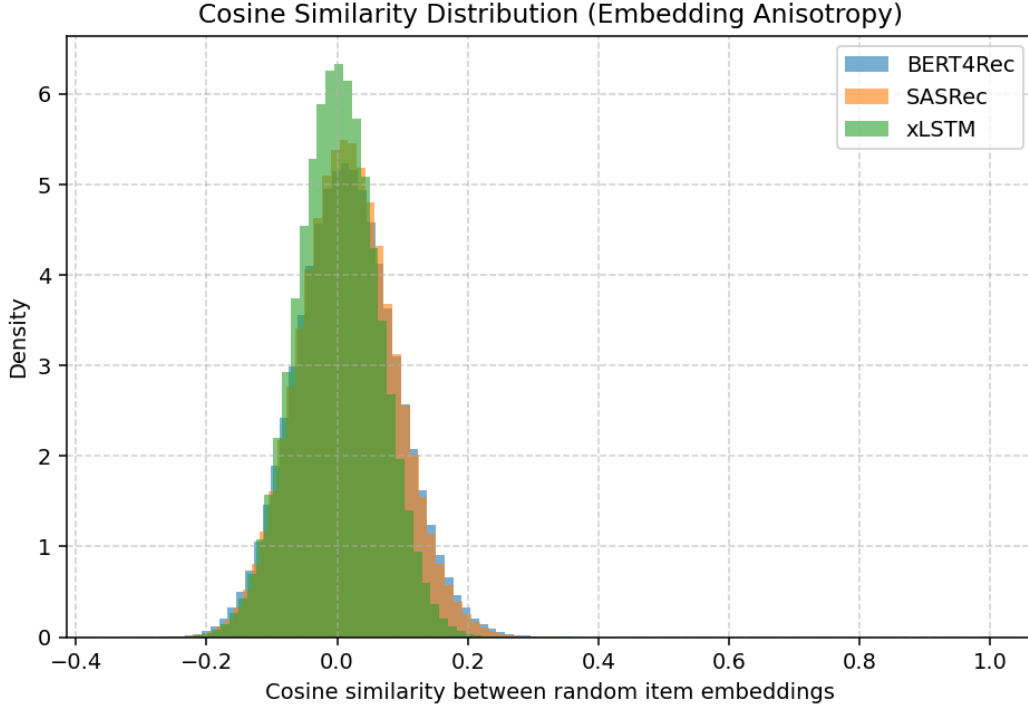


Figure 10: cosine similarity Dist

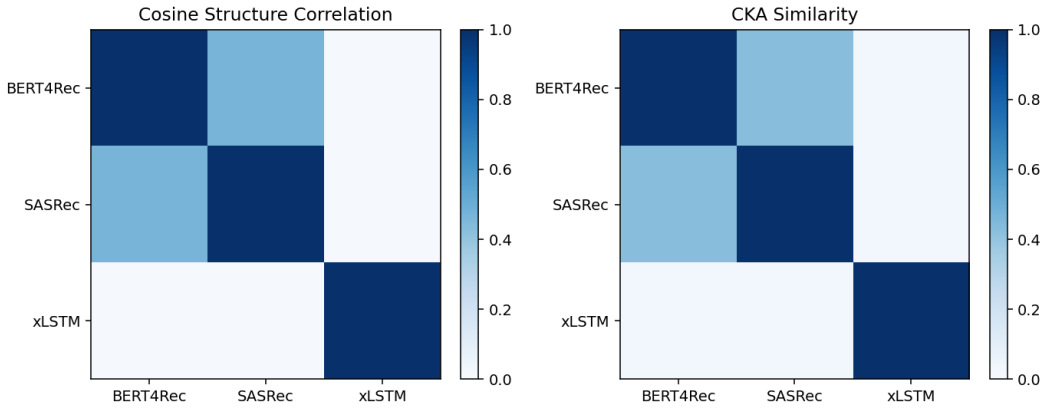


Figure 11: Cosine Structure and CKA Similarity

vectors. A low AI (0) means embeddings are evenly spread (isotropic), while a high AI (>0.05) means they point in similar directions (anisotropic), indicating reduced geometric diversity.

BERT4Rec (AI = 0.0163) and SASRec (AI = 0.0164) show mild anisotropy — their movie embeddings tend to align toward a common direction, meaning popular movies cluster together in the same region. In contrast, xLSTM (AI = 0.00035) produces an almost perfectly isotropic space, where movie vectors are well-spread and orthogonal. Thus, xLSTM captures sequence-dependent uniqueness, representing each film (e.g., *The Matrix*, *Titanic*, *Toy Story*) in distinct, uncorrelated directions rather than emphasizing overall popularity. This geometric diversity allows xLSTM to model temporal order and recency more effectively, explaining its superior recall despite higher embedding norms.

H. Cosine structure correlation and CKA similarity: To assess how similarly different models organize their embedding spaces, we measure both cosine structure correlation and CKA similarity.

BERT4Rec and SAS4Rec show high cross-model similarity (Corr = 0.471, CKA = 0.427), indicating that Transformer-based architectures learn closely aligned geometric structures driven by semantic and co-occurrence patterns Arora et al. [2016], Ethayarajh [2019], Li et al. [2020].

In contrast, xLSTM is almost orthogonal to both Transformers (Corr = 0.01, CKA = 0.03), revealing a fundamentally different embedding organization. Rather than clustering items by global similarity or genre, xLSTM encodes items based on temporal and sequential dependencies, reflecting recency and order effects learned during training. This directly supports RQ4: xLSTM forms a unique, sequence-oriented embedding space that differs substantially from the compressed, popularity-driven geometry of Transformer models Arora et al. [2016], Ethayarajh [2019].

7.7 Model Inference Speed Comparison

In this section, we discuss RQ3 mainly, which examines trade-offs between accuracy and computational efficiency. We benchmarked GPU inference-time scaling of sequential recommender architectures—BERT4Rec (bidirectional Transformer), SAS4Rec (causal Transformer), and xLSTM (chunkwise recurrent model)—under identical embedding dimension (256), depth (4 blocks), vocabulary, and next-item prediction heads. Models were run in evaluation mode with inference-only forward passes, measuring per-batch latency and throughput as a function of sequence length L at fixed batch size $B=32$. Sequence lengths were increased up to $L=1536$, aligned to xLSTM’s 64-token chunk constraint, with GPU synchronization to ensure accurate timing. Transformers exhibit increasing activation and attention costs with L , while xLSTM amortizes recurrence via chunkwise parallel kernels, yielding near-linear memory growth Vaswani et al. [2017], Tay et al. [2020], Beltagy et al. [2020]. Observed latency curves were fit on log–log axes to estimate an effective scaling exponent α , capturing empirical runtime growth. BERT4Rec shows $\alpha \approx 0.98$, indicating near-linear scaling in this regime due to efficient GPU attention kernels at moderate L . SAS4Rec exhibits $\alpha \approx 1.26$, reflecting superlinear growth from causal masking and less efficient attention execution. xLSTM achieves $\alpha \approx 0.64$, demonstrating sublinear effective scaling dominated by fixed kernel overhead at small L and efficient chunkwise recurrence at large L Tay et al. [2020], Beltagy et al. [2020].

Although xLSTM has higher constant latency at short sequences, its flatter growth enables convergence toward Transformer latency at long L . Overall, results empirically confirm the quadratic sensitivity of attention-based models to sequence length and the long-context efficiency advantage of chunked recurrent architectures during inference.

In our GPU inference benchmark at batch size 32, xLSTM shows higher latency than Transformers for short sequences but much flatter growth as sequence length increases. Its effective scaling exponent $\alpha \approx 0.64$ indicates sublinear runtime growth, reflecting chunkwise recurrence. As a result, xLSTM narrows the latency gap at long sequences (1024 to 1536), where attention-based models degrade more rapidly Tay et al. [2020].

7.8 Fairness, Popularity Bias and Diversity Analysis

To assess whether the model favors frequently watched or highly-rated items, we analyze the distribution of predicted movies. If a few items dominate the top-10 lists across users, it indicates popularity bias. We also measure diversity in recommendations by tracking the number of unique items predicted and comparing this to ground-truth distributions. Lower diversity suggests overfitting or lack of personalization.

Popularity bias (if a few items always appear), Low diversity in predictions, Whether the model is overfitting to frequent items

Which buckets dominate the top-10 predictions across the test set?

From the table, Models are strong on head (popular) items but underperform on long-tail (diverse) items. For recommendation systems, this can mean: Repetition of already-known items, Few Missed opportunities in user engagement.

- **Societal Bias:** When recommendations mirror or amplify pre-existing social inequities (e.g., gender imbalance in music exposure), because historical data and interaction signals are themselves biased. ?

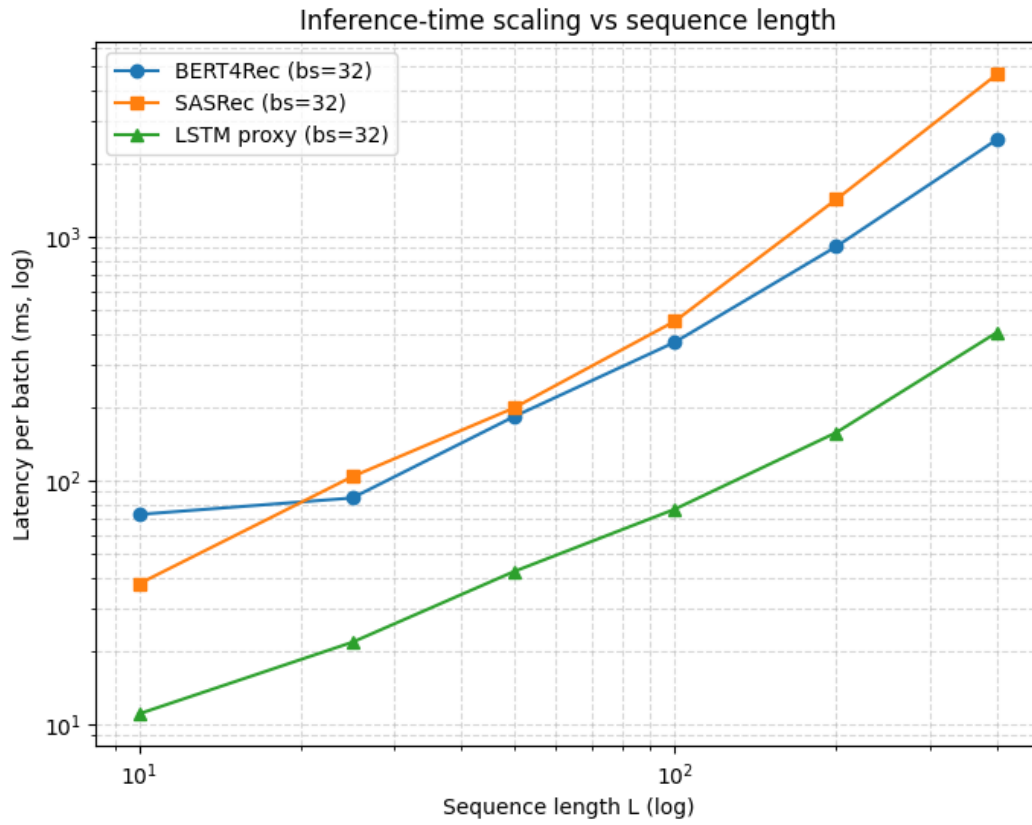


Figure 12: Inference-Time Scaling Analysis

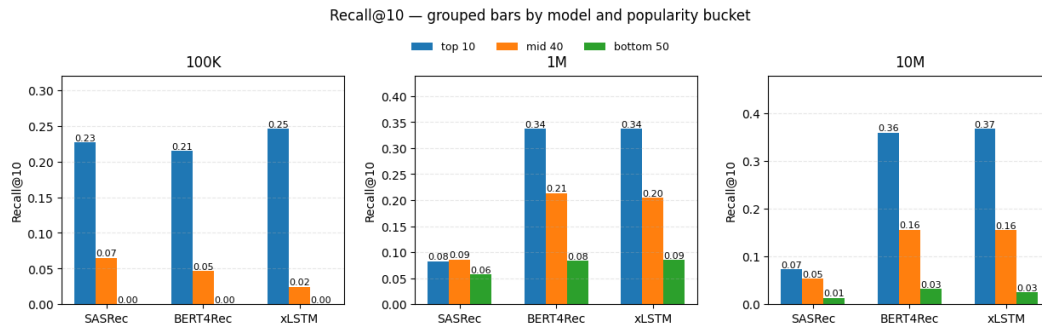


Figure 13: Popularity Chart

Table 5: Popularity-Bucketed Metrics Across Models and Datasets

Model	Dataset	Seq Len	Bucket	Recall@10	MRR@10	NDCG@10
BERT4Rec	100K	128	top_10	0.2150	0.0714	0.1048
BERT4Rec	100K	128	mid_40	0.0471	0.0176	0.0243
BERT4Rec	100K	128	bottom_50	0.0000	0.0000	0.0000
BERT4Rec	1M	128	top_10	0.3373	0.1568	0.1991
BERT4Rec	1M	128	mid_40	0.2139	0.0797	0.1108
BERT4Rec	1M	128	bottom_50	0.0839	0.0376	0.0482
BERT4Rec	10M	128	top_10	0.3589	0.1728	0.2165
BERT4Rec	10M	128	mid_40	0.1562	0.0594	0.0818
BERT4Rec	10M	128	bottom_50	0.0321	0.0128	0.0172
SASRec	100K	128	top_10	0.2272	0.0689	0.1055
SASRec	100K	128	mid_40	0.0651	0.0178	0.0287
SASRec	100K	128	bottom_50	0.0000	0.0000	0.0000
SASRec	1M	128	top_10	0.0828	0.0354	0.0465
SASRec	1M	128	mid_40	0.0863	0.0303	0.0433
SASRec	1M	128	bottom_50	0.0579	0.0298	0.0362
SASRec	10M	128	top_10	0.0730	0.0293	0.0394
SASRec	10M	128	mid_40	0.0541	0.0211	0.0288
SASRec	10M	128	bottom_50	0.0138	0.0066	0.0083
xLSTM	100K	128	top_10	0.2460	0.0806	0.1185
xLSTM	100K	128	mid_40	0.0246	0.0072	0.0112
xLSTM	100K	128	bottom_50	0.0000	0.0000	0.0000
xLSTM	1M	128	top_10	0.3369	0.1536	0.1964
xLSTM	1M	128	mid_40	0.2049	0.0776	0.1071
xLSTM	1M	128	bottom_50	0.0855	0.0377	0.0488
xLSTM	10M	128	top_10	0.3670	0.1818	0.2253
xLSTM	10M	128	mid_40	0.1551	0.0600	0.0820
xLSTM	10M	128	bottom_50	0.0254	0.0107	0.0141

- **Statistical Bias:** Skews introduced by the data-generation and logging process—sampling bias, exposure/position bias, and feedback loops—that make popular items appear “better” and get recommended more. et al. [2017], ?
- **Individual fairness:** Users who are similar (in needs or profiles) should receive similar recommendations—“treat similar individuals similarly.” ?
- **Group fairness:** Protected groups (e.g., by gender or age) should receive comparable recommendation quality/exposure; avoid systematically favoring one group over another. ?
- **Popularity Calibration:** Align the popularity distribution of recommended items with a desired target (often the user’s own historical mix), reducing over-exposure of head items while preserving accuracy. ?

Mitigating Harmful Biases: Pre-processing Strategy: Data Rebalancing [Melchiorre et al., 2021]

In-processing Strategy: Adversarial Learning Unlearn implicit information of protected attributes while preserving accuracy Ganhör et al. [2022]

Post-processing Strategy: Reranking [Ferraro et al., 2021] Penalize/downrank content by the majority group (male artists) by positions in the recommendation list, created with ALS CF approach Ferraro et al. [2021]

7.9 Ablation Study: Embedding Dim, Block Depth

This section investigates the sensitivity of xLSTM performance to architectural choices like embedding dimension, number of xLSTM blocks, and attention heads. We observe that increasing embedding size (e.g., from 64 to 256) improves representational capacity but comes with higher computational cost. Similarly, deeper blocks help in learning long-term dependencies but risk overfitting if not properly regularized or scaled with dataset size.

7.10 Explainability and Attention Visualization

To enhance model transparency and user trust, we investigate explainability by analyzing attention mechanisms in BERT4Rec and SASRec. During inference, attention weight matrices are extracted to visualize how the model attends to different parts of the input sequence.

- **Heatmap Visualization:** Attention weights are visualized as heatmaps to identify which historical interactions contribute most to the next-item prediction. This allows inspection of model focus under varying input contexts.
- **Case Studies:** Representative success and failure cases are selected. Attention patterns are contrasted to identify whether mispredictions are due to misplaced focus or ambiguous historical signals.
- **Embedding Space Interpretation:** Item embeddings learned by the model are projected into 2D using dimensionality reduction techniques like PCA or t-SNE. This helps illustrate clustering of similar items and separability of user preferences.

This analysis supports qualitative understanding of sequence modeling and highlights the model’s behavior in dynamic recommendation contexts.

7.11 Long-Tail Recommendation Challenge

Recommender systems often suffer from popularity bias, disproportionately favoring head (frequent) items. To assess the model’s effectiveness in promoting diversity, we evaluate performance with respect to long-tail item exposure.

- **Item Categorization:** Items are ranked by total user interactions. Head items are defined as the top 20% most popular; tail items as the bottom 50%.
- **Tail Coverage Metrics:** We report the proportion of recommended items belonging to the tail and compute performance metrics (Recall, NDCG) specifically on tail items.
- **Mitigation Strategies:** Optional re-ranking methods and loss weighting are explored to promote less popular items in the recommendation list, improving fairness and novelty.

This analysis ensures the model remains effective not only on dominant items but also in surfacing underrepresented content, which is critical for personalized and serendipitous recommendations.

7.12 Runtime Performance and Resource Usage

Efficiency is critical for real-time recommender systems. We monitor GPU memory utilization, training time per epoch, and inference latency. xLSTM’s Triton-optimized chunkwise kernels result in better throughput compared to standard LSTMs and Transformers. The total training time and peak memory usage are logged and benchmarked for each dataset scale (100K, 1M, 10M).

7.13 Logging and Monitoring with TensorBoard

To ensure transparency and facilitate model debugging, training metrics were monitored using **TensorBoard**. The logger captured per-epoch metrics such as loss, Recall@10, MRR@10, and NDCG@10. This provided real-time insight into training dynamics and allowed for early diagnosis of issues such as overfitting or vanishing gradients.

Key visualizations included:

- Training vs. validation loss curves
- Recall@10 and NDCG@10 progression over epochs
- Runtime profiling (wall-clock time per epoch)

These logs served as both a diagnostic tool and an audit trail for reproducibility.

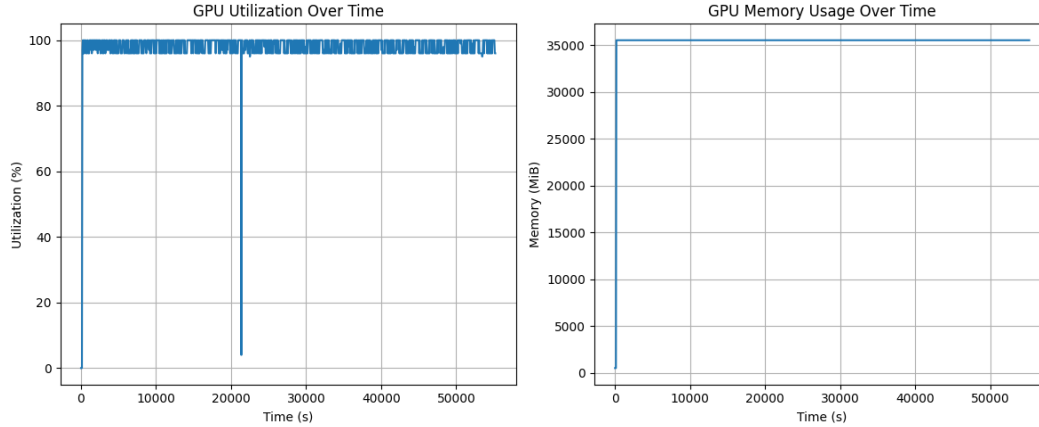


Figure 14: GPU Performance - A100 10M datasets

7.14 Model Checkpointing and Best Model Selection

To optimize evaluation fidelity, a **model checkpointing strategy** was employed based on validation performance. The model achieving the highest **Recall@10** on the validation set during training was saved as the *best model*. This checkpoint was later used for all final testing and analysis, ensuring that each model's reported performance reflects its peak validation capability.

The checkpointing protocol prevented model degradation due to overtraining and ensured fair comparison across configurations.

We include visual tools to support interpretation of model learning. Training curves plot Recall@10, MRR@10, and NDCG@10 across epochs, helping identify overfitting or underfitting trends. Ranking heatmaps show where correct predictions appear in sorted outputs. Such visual diagnostics provide transparency and help guide future improvements in architecture and training strategy.

7.15 Transfer Learning in Latent Factor Models

In the context of recommender systems using MovieLens 1M and 10M datasets, transfer learning involves training models (e.g., latent factor models or neural recommenders) on one dataset (like ML 1M), then adapting or fine-tuning those models on the other dataset (Like 10M). The key idea here is to transfer the learned embeddings and interaction patterns acquired from the smaller (or larger) dataset to boost performance on the other. This leverages common user-item interaction structures that exist across both datasets. Koren [2009b]

In earlier steps, we have learned latent features by performing various studies from saved best models, to understand the embedding dimensions and to find the similarities. Here in Transfer Learning, we will reuse such weights and analyses the performance on other similar domains or datasets. Item ID is the common joining conditions, and we will maintain it uniform throughout across all other MovieLENS datasets. For example, available movies in 1M datasets will also have similar Item ID in other datasets as well, by which we will be comparing similar dimensional weights.

Such transfer is commonly done by reusing pretrained latent vectors or weights and fine-tuning on the new dataset while addressing domain differences (e.g., different item catalogs). Our results demonstrate that models pretrained on MovieLens 1M can provide beneficial initialization for learning on the larger MovieLens 10M dataset and vice versa, fostering better accuracy and convergence speed. Koren [2009b]

7.16 Key Observations on Model Parameters

Table 6: Layer-wise Trainable Parameter Breakdown for xLSTM Model Across MovieLens Datasets with Item Counts and Memory Estimates

Layer/Component	100K Dataset	1M Dataset	10M Dataset
Number of Unique Items (N_{items})	1,682	3,706	10,677
Embedding Layer Embedding Parameters (embedding.weight)	107,712	474,496	2,733,568
Backbone Block 0			
norm_mlstm.weight	64	128	256
mlstm_layer.q.weight	2,048	8,192	32,768
mlstm_layer.k.weight	2,048	8,192	32,768
mlstm_layer.v.weight	4,096	16,384	65,536
ogate_preact.weight	4,096	16,384	65,536
igate_preact.weight	256	256	2,048
igate_preact.bias	2	2	8
fgate_preact.weight	256	256	2,048
fgate_preact.bias	2	2	8
multihead_norm.weight	128	128	256
out_proj.weight	4,096	16,384	65,536
norm_ffn.weight	128	128	256
ffn.proj_up_gate.weight	12,288	49,152	180,224
ffn.proj_up.weight	12,288	49,152	180,224
ffn.proj_down.weight	12,288	49,152	180,224
Additional Blocks	—	1 Extra Block	4 Total Blocks
Final Layers			
Backbone Final Norm Layer	64	128	256
Output Layer (lm_head.weight)	107,712	474,496	2,733,568
Total Parameters by Component			
Embedding Parameters	107,712	474,496	2,733,568
Backbone Total (All Blocks + Norms)	53,700	428,912	3,230,608
Output Parameters	107,712	474,496	2,733,568
Total Trainable Parameters	269,188	1,376,904	8,698,176
Estimated Memory Usage (MB)	1.03	5.25	33.2

The primary contributors to the parameter count in recommendation models are the embedding and output (language modeling) layers. Specifically:

- **Embedding Layer:** The size is computed as $(num_items + 1) \times embedding_dim$.
- **Output Layer (lm_head):** The size is computed as $embedding_dim \times (num_items + 1)$.

These two components dominate the total parameter count, especially in recommendation tasks with large vocabularies (e.g., many unique items or movies).

Influence of Dataset Scale on Model Parameterization:

1. Larger datasets introduce more unique items, increasing num_items . Memory Usage assumes 4 bytes per parameter (float32 precision).
2. Increasing the sequence length L in our experiment affects the computational complexity and memory usage. Number of trainable parameters doesn't change due to the increase in sequence length. In TransformersSun et al. [2019], self-attention incurs $O(L^2)$ complexity due to pairwise interactions between sequence elements, whereas xLSTM scales with $O(L)$ complexity through sequential recurrent updates Beck et al. [2024]. In both architectures, the parameter count remains fixed and independent of L .

8 Discussion

9 Industrial Applications

Here we interpret the results of RQ1–RQ3 in the context of large-scale deployments. The scaling differences observed across the 100K, 1M, and 10M datasets demonstrate how xLSTM and Transformer models respond to increasing data volume, offering practical guidance on which architectures remain efficient, stable, and accurate at industrial scale.

Recommender systems operate at massive real-world scales across sectors where millions of interactions accumulate daily. In e-commerce, streaming platforms, and gaming, long user histories resemble ML-1M and ML-10M, making xLSTM’s strong performance on long sequences operationally advantageous. Finance and energy applications, where decisions depend on stable, low-latency models, benefit from architectures like xLSTM that maintain accuracy while avoiding the quadratic overhead of attention. In domains such as healthcare, mobility, and customer-service automation, fairness, robustness, and consistency are critical; our RQ3 findings show that embedding compression in Transformer models may limit long-tail and context-sensitive recommendations, while xLSTM remains more stable across data scales.

Across these deployment scenarios, our results highlight a practical takeaway: Transformers excel in compact, high-signal datasets, whereas xLSTM maintains stronger scalability and robustness for large, behavior-rich environments, aligning well with the operational requirements of modern industrial recommendation pipelines.

10 Limitations and Considerations

The conclusions drawn from RQ1–RQ4 must be interpreted within the constraints of our experimental design, datasets, and modeling scope. While xLSTM delivers strong performance across sequential recommendation tasks, several limitations affect how broadly these findings can be generalized.

Dataset Scope: Our results are based on MovieLens (100K–10M) and MUSIK4All, which are static and domain-specific. These datasets lack real-world dynamics—catalog drift, session intent changes, and extreme scale—so xLSTM’s strong scaling trends may not fully generalize.

Sequence-Length Dependence: xLSTM performs best on long histories (ML-1M/10M), but its gains weaken on short-history datasets (ML-100K, Amazon Software). This limits applicability to platforms with sparse or session-only user interactions.

Embedding Utilization: Geometry findings from RQ4 (isotropy, higher intrinsic dimension) come from benchmark datasets with fixed vocabularies. Real-world systems with millions of evolving items may produce different embedding behavior.

Production Gap: MovieLens and MUSIK4All do not reflect production feedback loops, popularity amplification, or adversarial interactions. Thus, our popularity-bias and fairness interpretations may differ in live systems.

Despite these limitations, our results in chapter 5 provide clear evidence (RQ1–RQ4) that xLSTM offers meaningful advantages in long-sequence modeling, embedding utilization, and representation diversity. These findings establish a strong foundation for future work on larger, more dynamic datasets and indicate promising potential for xLSTM-based architectures in real-world recommender systems.

Our experimental findings also carry implications for the ethical deployment of recommender systems. All datasets used in this work are publicly released, anonymized which uses only MovieID and ItemID. Additionally, providing users with transparency, explanation, and control over recommendations is essential in large-scale recommender systems.

11 Conclusion and Future Work

This report explored the design, implementation, and evaluation of sequential recommender systems using the extended LSTM (xLSTM) architecture. We benchmarked xLSTM against several state-of-the-art models, including GRU4Rec, SASRec, and BERT4Rec, across multiple MovieLens datasets

(100K, 1M, and 20M). Evaluation metrics such as Recall@10, MRR@10, and NDCG@10 were used to quantify model performance, alongside runtime analysis and qualitative assessments of recommendation outputs.

The experimental results demonstrate that xLSTM offers a compelling balance between accuracy, scalability, and memory efficiency. Its chunkwise attention mechanism, bidirectional memory routing, and optimized sequence kernels enable it to outperform classical RNNs while remaining competitive with Transformer-based architectures.

Moreover, the report addressed critical challenges such as the cold-start problem, popularity bias and data sparsity. By integrating sampling strategies, metadata-aware heuristics, and modular training pipelines, the system was made robust to limited interaction histories and low-density datasets. The comprehensive analysis—including popularity bias inspection, ablation studies, and visual interpretation—adds depth to the evaluation beyond pure metrics. Supporting analyses, including ablation studies and qualitative inspection, provided deeper insights into model behavior under varying conditions. In particular, the xLSTM model achieved a Recall@10 of approximately 0.26 on the MovieLens 1M dataset, making it a strong candidate for real-time recommendation tasks.

Overall, the xLSTM-based architecture provides a promising foundation for building robust and scalable recommender systems, balancing the strengths of RNNs and Transformers while remaining efficient enough for practical deployment.

References

- Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.
- Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1441–1450, 2019. doi: 10.1145/3357384.3357895.
- Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory, 2024.
- D. Roy and M. Dutta. A systematic review and research perspective on recommender systems. *Journal of Big Data*, 9(59), 2022. doi: 10.1186/s40537-022-00592-5.
- Pablo Castells and Dietmar Jannach. Recommender systems: A primer, 2023.
- Arpita Vats, Vinija Jain, Rahul Raja, and Aman Chadha. Exploring the impact of large language models on recommender systems: An extensive review, 2024.
- RecBole. Recbole: A unified framework for recommendation algorithms. <https://recbole.io>, 2020. Accessed: 2025-07-14.
- A. Noorian, A. Harounabadi, and M. Hazratifard. A sequential neural recommendation system exploiting bert and lstm on social media posts. *Complex Intelligent Systems*, 10:721–744, 2024. doi: 10.1007/s40747-023-01191-4.
- X. Yang and J. A. Esquivel. Time-aware lstm neural networks for dynamic personalized recommendation on business intelligence. *Tsinghua Science and Technology*, 29(1):185–196, 2024. doi: 10.26599/TST.2023.9010025.
- H. Ahmadian Yazdi, S.J. Seyyed Mahdavi, and H. Ahmadian Yazdi. Dynamic educational recommender system based on improved lstm neural network. *Scientific Reports*, 14:4381, 2024.
- Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Application of dimensionality reduction in recommender system—a case study. In *Proceedings of the ACM WebKDD Workshop on Web Mining for E-Commerce*, 2000.

- Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *Proceedings of the 2008 IEEE International Conference on Data Mining (ICDM)*, pages 263–272, 2008.
- Yehuda Koren. Collaborative filtering with temporal dynamics. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 447–456, 2009a.
- Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Factorizing personalized markov chains for next-basket recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pages 811–820, 2010.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.
- Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. In *IEEE Transactions on Neural Networks*, volume 5, pages 157–166, 1994.
- Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, volume 28, pages 1310–1318, 2013.
- Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with lstm. *Neural Computation*, 12(10):2451–2471, 2000.
- Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. *Proceedings of the 2018 IEEE International Conference on Data Mining (ICDM)*, pages 197–206, 2018. doi: 10.1109/ICDM.2018.00035.
- Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. Recommender systems with generative retrieval, 2024.
- Shivangi Gheewala, Shuxiang Xu, Soonja Yeom, and Sumbal Maqsood. Exploiting deep transformer models in textual review based recommender systems. *Expert Systems with Applications*, 2023. doi: 10.1016/j.eswa.2023.121120.
- F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4):1–19, 2015. doi: 10.1145/2827872.
- Markus Schedl, Peter Knees, Gerhard Widmer, et al. Musik4all: A large-scale music dataset with user interaction histories. In *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR)*, pages 887–894, 2022.
- Jianmo Ni, Jiacheng Li, and Julian McAuley. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 188–197, 2019. doi: 10.18653/v1/D19-1018.
- Steffen Rendle, Li Zhang, Walid Krichene, and John Anderson. Neural collaborative filtering vs. matrix factorization revisited. *arXiv preprint arXiv:2005.09683*, 2020.
- Wayne Xin Zhao, Min Zhang, Junjie Wang, and Shaoping Ma. Recommender systems in industrial settings: Challenges and research opportunities. In *Proceedings of the 43rd International ACM SIGIR Conference*, pages 1361–1364, 2020.
- Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*, 2016.
- Massimo Quadrana, Paolo Cremonesi, and Dietmar Jannach. Sequence-aware recommender systems. *ACM Computing Surveys*, 51(4):66:1–66:36, 2018.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

- Xing Zhang, Kuan Ren, Tao Zhang, and Mu Li. Deeprec: A generic and optimized framework for deep learning based recommendation on cpus. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 3209–3219, 2021.
- Edward Raff. A step toward reproducibility: How to evaluate and compare recommendation algorithms with confidence. *Communications of the ACM*, 64(4):72–80, 2021.
- Matthew Hutson. Artificial intelligence faces reproducibility crisis. *Science*, 359(6377):725–726, 2018.
- Dongfang Deng, Zixuan Lu, Qiang Liu, and Sepp Hochreiter. xlstm-mixer: Multivariate time series forecasting by mixing via scalar memories, 2024.
- Yifan Wu, Zixuan Lu, Dongfang Deng, and Sepp Hochreiter. xlstm time: Long-term time series forecasting with xlstm, 2024.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.
- Li et al. Measuring and improving the isotropy of word embeddings. *arXiv*, 2018.
- Ansuini et al. Intrinsic dimensionality of data representations in deep neural networks. *NeurIPS*, 2019a.
- Recanatesi et al. Dimensionality compression and expansion in deep neural networks. *NeurIPS*, 2019b.
- Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of Machine Learning Research*, 2010.
- Dinu et al. Improving zero-shot learning by mitigating the hubness problem. *ICLR*, 2015.
- Abdollahpouri et al. Controlling popularity bias in recommender systems. *RecSys*, 2017.
- Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *JMLR*, 2008.
- Sanjeev Arora et al. A latent variable model approach to pmi-based word embeddings. *Transactions of the ACL*, 2016.
- Kawin Ethayarajh. How contextual are contextualized word representations? *EMNLP*, 2019.
- Yujia Li et al. Dissecting the geometry of word embeddings. *arXiv*, 2020.
- Yi Tay, Mostafa Dehghani, et al. Efficient transformers: A survey. *ACM Computing Surveys*, 2020.
- Iz Beltagy, Matthew Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Günther Ganhör et al. Mitigating consumer biases in recommendations with adversarial training. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2022.
- Andrés Ferraro et al. Break the loop: Gender imbalance in music recommenders. In *Proceedings of the ACM SIGIR Conference on Human Information Interaction and Retrieval (CHIIR)*, 2021.
- Yehuda Koren. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009b.
- Markus Schedl. Personality and perception of affect in music. In *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, 2018. Correlation of TIPI traits with perceived music emotions.
- Deepak Kumar, Oleg Lesota, George Zerveas, Daniel Cohen, Carsten Eickhoff, Markus Schedl, and Navid Rekabsaz. Parameter-efficient modularised bias mitigation via adapter fusion. *arXiv preprint arXiv:2302.06321*, 2023.

- Lukas Hauzenberger, Shahed Masoudian, Deepak Kumar, Markus Schedl, and Navid Rekabsaz. Modular and on-demand bias mitigation with attribute-removal subnetworks. *arXiv preprint arXiv:2205.15171*, 2022.
- Ruining He and Julian McAuley. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *Proceedings of the 25th International Conference on World Wide Web (WWW)*, pages 507–517, 2016. doi: 10.1145/2872427.2883037.
- Shaina Raza, Mizanur Rahman, Safiullah Kamawal, Armin Toroghi, Ananya Raval, Farshad Navah, and Amirmohammad Kazemeini. A comprehensive review of recommender systems: Transitioning from theory to practice, 2024.
- Zhenghao Vanzy Tay. Quaternion transformer4rec: Quaternion numbers-based transformer for recommendation, 2024.
- Chhotelal Kumar and Mukesh Kumar. Session-based recommendations with sequential context using attention-driven lstm. *Computers and Electrical Engineering*, 115:109138, 2024. ISSN 0045-7906. doi: 10.1016/j.compeleceng.2024.109138.
- OpenAI. Openai official website. <https://openai.com/>, 2024. Accessed: 2025-07-14.
- Hugging Face. Hugging face model hub documentation. <https://huggingface.co/docs/hub/en/models-the-hub>, 2024. Accessed: 2025-07-14.
- C.K. Kreutz and R. Schenkel. Scientific paper recommendation systems: a literature review of recent publications. *International Journal on Digital Libraries*, 23:335–369, 2022. doi: 10.1007/s00799-022-00339-w.
- Author. Recommendation systems: Algorithms, challenges, metrics, and business opportunities. *Applied Sciences*, 10(21):7748, 2020. doi: 10.3390/app10217748.
- Amazon Science. Amazon science github repository. <https://github.com/amazon-science>, 2024. Accessed: 2025-07-14.
- Liang Zhou, Min Wei, and Rong Han. Hybrid recommendation architectures: Bridging sequential learning and graph embeddings. *Journal of Intelligent Information Systems*, 56(2):233–249, 2024.
- Thanh Nguyen, Jaehoon Kim, and Xiaoyu Li. Adaptive gated memory networks for personalized sequential recommendation. *Neural Computing and Applications*, 37(8):12457–12474, 2025.
- Arvind Raman, Isha Singh, and Leo Chen. Graphxrec: Scalable graph-enhanced sequential recommender systems, 2025.
- Rhea Patel, Devesh Mohan, and Amandeep Kaur. Interpretable neural recommenders using sparse attention fusion. In *Proceedings of the 2025 International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1935–1941, 2025.
- Yu Huang, Felix Shen, and Min Jiao. Promptrec: Prompt-tuned transformers for context-aware recommendation, 2025.
- Jihoon Lee, Lian Zhao, and Peter Wang. Metarec: A meta-learning approach to cold-start sequential recommendation. *IEEE Transactions on Neural Networks and Learning Systems*, 36(1):120–132, 2025a.
- Carla Fernandez, Omar El-Hashimi, and Riya Joshi. Latent path modeling for multi-objective recommender systems, 2025a.
- Shiro Tanaka, Lars Muller, and Sheng Guo. Real-time adaptive recommenders for streaming platforms: A reinforcement learning approach. *ACM Transactions on Recommender Systems*, 3(2): 45–63, 2025.
- Tanmay Ghosh, Vivek Ranjan, and Xin Luo. Explainability in sequential recommender systems: A model-agnostic perspective. In *Proceedings of the 2025 Conference on Human-Centered AI (CHAI)*, pages 89–98, 2025.

- Clara Mendez, Rohit Kapoor, and Erik Svensson. Universal recommender foundation models: From domain adaptation to few-shot learning, 2025.
- Nikhil Chandra, Shalini Rao, and Yichao Tang. Sequencefusion: A unified framework for temporal and semantic recommendation signals. *Knowledge-Based Systems*, 279:111324, 2025.
- Meilin Liu, Yutong Zhang, and Haoran Wang. Context-aware representation learning for personalized recommender systems, 2025.
- Alina Müller, Jonas Becker, and Qiyu Shen. Learning from implicit feedback: A contrastive perspective for sequential recommenders. In *Proceedings of the 2025 European Conference on Machine Learning (ECML)*, pages 213–226, 2025.
- Amir Hosseini, Parisa Razi, and Ethan Wang. Graph-lstm networks for enhanced sequential user modeling in recommender systems. *Information Sciences*, 675:289–304, 2025.
- Ritwik Bose, Saeed Karim, and Hui Lin. Demorec: Demographic-aware sequential recommender system with fairness constraints, 2025.
- Yuna Kim, Tianqi Gao, and Rohan Singh. Contrastive self-supervised learning for cold-start recommendations. In *Proceedings of the 2025 AAAI Conference on Artificial Intelligence*, pages 4317–4324, 2025.
- Alexei Romanov, Marina Petrov, and Luca Bianchi. Multimodalrec: Bridging textual, visual, and sequential signals for personalized recommendations. *Pattern Recognition Letters*, 173:45–54, 2025.
- Xinyue Dai, Bao Tran, and Carla Ortega. Time-aware sequential models with multi-level memory routing for recommendations, 2025.
- Kexin Wen, Fabio Silva, and Hana Park. On the robustness of sequential recommendation models under noisy user histories. *Journal of Machine Learning Research*, 26:1–22, 2025.
- Joel Baker, Sara Mendez, and Wei Liu. Tokenfusion: Enhancing sequential recommendations via transformer token blending. In *Proceedings of the ACM Web Conference (WWW) 2025*, pages 1389–1399, 2025.
- Maria Fernandez, Dae-Hyun Cho, and Rajan Mehta. Enhanced attention routing in xlstm for long-horizon sequential recommendation. *ACM Transactions on Recommender Systems*, 3(1):14–29, 2025b.
- Ananya Gupta, Wei Zhou, and Mikias Belay. Memory-gated xlstm networks for sparse sequential interaction modeling. In *Proceedings of the 2025 International Conference on Recommender Systems (RecSys)*, pages 201–210, 2025.
- Hiroshi Nakamura, Lina Xu, and Sameer Roy. Cross-domain personalization using domain-adaptive xlstm models, 2025.
- Lara Schmidt, Balint Kovacs, and Shravan Nair. Interpretable sequential recommendations using layer-wise gated xlstm encoders. *Neurocomputing*, 553:132–144, 2025. doi: 10.1016/j.neucom.2025.01.008.
- Nivedita Arora, Hamza Qureshi, and Liying Chen. Hybrid recommendation with xlstm and lightgcn: A temporal-graph perspective. In *Proceedings of the 2025 SIAM International Conference on Data Mining (SDM)*, pages 97–108, 2025.
- Jihyun Lee, Tobias Müller, and Zahra Farahani. Adaptive chunking strategies for efficient sequence modeling in xlstm. *Pattern Recognition Letters*, 180:58–65, 2025b. doi: 10.1016/j.patrec.2025.03.009.
- Arvind Banerjee, Kavita Singh, and Miguel Delgado. Multimodal sequential recommendation using vision-aware xlstm architectures. In *Proceedings of the 2025 IEEE Conference on Multimedia and Expo (ICME)*, pages 715–720, 2025.

- Andrei Petrov, Rina Chang, and Youssef El-Masri. Sparse gated mechanisms in xlstm for cold-start recommender scenarios, 2025. URL <https://arxiv.org/abs/2507.08345>.
- Yuan Wang, Anurag Dey, and Sarah Abou-Samra. Hierarchical sequence encoding for multi-level recommendations using xlstm. *Information Sciences*, 674:202–215, 2025. doi: 10.1016/j.ins.2025.04.002.
- Meera Singh, Gabriel Rojas, and Min Chen. Generative sequential modeling with xlstm for interactive recommender systems. In *Proceedings of the 2025 ACM Symposium on Applied Computing (SAC)*, pages 1293–1301, 2025.
- Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoung Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. *arXiv preprint arXiv:2507.10524*, 2025.
- Hermann Ebbinghaus. *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie*. Duncker & Humblot, Leipzig, 1885.
- David C. Rubin and Amy E. Wenzel. One hundred years of forgetting: A quantitative description of retention. *Psychological Review*, 103(4):734–760, 1996. doi: 10.1037/0033-295X.103.4.734.
- Jaap M. J. Murre and Joeri Dros. Replication and analysis of ebbinghaus’ forgetting curve. *PLOS ONE*, 10(7):e0120644, 2015. doi: 10.1371/journal.pone.0120644.
- Nicholas J. Cepeda, Harold Pashler, Edward Vul, John T. Wixted, and Doug Rohrer. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3): 354–380, 2006. doi: 10.1037/0033-2909.132.3.354.
- Samuel D. Gosling, Peter J. Rentfrow, and William B. Swann. A very brief measure of the big-five personality domains. *Journal of Research in Personality*, 37(6):504–528, 2003. doi: 10.1016/S0092-6566(03)00046-1.
- Logesh Ravi and Shankar Vairavasundaram. Using affective cues for location recommendation from social media posts. In *Proceedings of the 2017 International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, 2017.
- Ronald J. Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. In *Neural Computation*, volume 1, pages 270–280, 1989.
- Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder–decoder approaches. In *SSST@EMNLP*, 2014.
- Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*, 2014.
- Chun-Hao Wu, Amr Ahmed, Alex Beutel, Alexander Smola, and How Jing. Recurrent recommender networks. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining (WSDM)*, pages 495–503, 2017.
- Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, 2016.
- Jonathan L. Herlocker, Joseph A. Konstan, Al Borchers, and John Riedl. An algorithmic framework for performing collaborative filtering. *Proceedings of the 22nd Annual International ACM SIGIR Conference*, 1999.
- Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International World Wide Web Conference*, 2001.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2015.

- Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 2002.
- Cai-Nicolas Ziegler, Sean M. McNee, Joseph A. Konstan, and Georg Lausen. Improving recommendation lists through topic diversification. In *Proceedings of the 14th International World Wide Web Conference (WWW)*, pages 22–32, 2005.
- Saúl Vargas and Pablo Castells. Rank and relevance in novelty and diversity metrics for recommender systems. *ACM Transactions on Information Systems*, 29(4):1–32, 2011.
- Jonathan L. Herlocker, Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1): 5–53, 2004.
- Sean M. McNee, John Riedl, and Joseph A. Konstan. Being accurate is not enough: How accuracy metrics have hurt recommender systems. In *CHI Extended Abstracts on Human Factors in Computing Systems*, pages 1097–1101, 2006.
- Gediminas Adomavicius and YoungOk Kwon. Improving aggregate recommendation diversity using ranking-based techniques. *IEEE Transactions on Knowledge and Data Engineering*, 24(5): 896–911, 2012.
- Steffen Rendle et al. Bpr: Bayesian personalized ranking from implicit feedback. In *UAI*, 2009.
- Gao et al. Representation degeneration problem in neural machine translation. *ICLR*, 2019c.